

Lab assignment-11.2

Name:Anand

Hallticket:2303A51090

Batch-02

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Prompt:

#generate a code to implement all stack operations like push , pop, peek, isEmpty and display using list in python

Code:

```

1  #generate a code to implement all stack operations like push , pop, peek, isEmpty and display using list in python
2  class Stack:
3      def __init__(self):
4          self.stack = []
5
6      def push(self, item):
7          """Add an item to the top of the stack."""
8          self.stack.append(item)
9          print(f"Item '{item}' pushed to stack.")
10
11     def pop(self):
12         """Remove and return the item at the top of the stack. Raises an error if the stack is empty."""
13         if not self.is_empty():
14             item = self.stack.pop()
15             print(f"Item '{item}' popped from stack.")
16             return item
17         else:
18             print("Stack is empty. Cannot pop item.")
19             return None
20
21     def peek(self):
22         """Return the item at the top of the stack without removing it. Raises an error if the stack is empty."""
23         if not self.is_empty():
24             item = self.stack[-1]
25             print(f"Item at top of stack: '{item}'")
26             return item
27         else:
28             print("Stack is empty. Cannot peek item.")
29             return None
30
31     def is_empty(self):
32         """Return True if the stack is empty, False otherwise."""
33         return len(self.stack) == 0
34
35     def display(self):
36         """Display all items in the stack."""
37         if not self.is_empty():
38             print("Stack contents:")
39             for item in reversed(self.stack):
40                 print(item)
41         else:
42             print("Stack is empty.")
43
44     # Example usage
45     if __name__ == "__main__":
46         stack = Stack()
47         stack.push(1)
48         stack.push(2)
49         stack.push(3)
50         stack.display()
51         stack.pop()
52         stack.display()
53         print("Is stack empty?", stack.is_empty())
54         stack.pop()
55         stack.pop()
56         print("Is stack empty?", stack.is_empty())
57

```

Output:

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Item '1' pushed to stack.
Item '2' pushed to stack.
Item '3' pushed to stack.
Stack contents:
3
2
1
Item at top of stack: '3'
Item '3' popped from stack.
Stack contents:
2
1
Is stack empty? False
Is stack empty? False
Item '2' popped from stack.
Item '1' popped from stack.
Is stack empty? False
Item '2' popped from stack.
Is stack empty? False
Item '2' popped from stack.
Item '1' popped from stack.
Is stack empty? False
Item '2' popped from stack.
Item '1' popped from stack.
Is stack empty? True
PS C:\Users\arell\Music\aiac>

Is stack empty? False
Item '2' popped from stack.
Item '1' popped from stack.
Is stack empty? True
Item '2' popped from stack.
Item '1' popped from stack.
Item '1' popped from stack.
Is stack empty? True
PS C:\Users\arell\Music\aiac>

```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:

#generate a code to implement queue operations like enqueue, dequeue, peek, isEmpty and display using list in python

Code:

```
53 
54 #generate a code to implement queue operations like enqueue, dequeue, peek, isEmpty and display using list in python
55 class Queue:
56     | Add to chat (Alt+X) | Do not show again
57     def __init__(self):
58         self.queue = []
59
60     def enqueue(self, item):
61         self.queue.append(item)
62         print(f"Item '{item}' enqueued to queue.")
63
64     def dequeue(self):
65         if not self.is_empty():
66             item = self.queue.pop(0)
67             print(f"Item '{item}' dequeued from queue.")
68             return item
69         else:
70             print("Queue is empty. Cannot dequeue item.")
71             return None
72
73     def peek(self):
74         if not self.is_empty():
75             item = self.queue[0]
76             print(f"Item at front of queue: '{item}'")
77             return item
78         else:
79             print("Queue is empty. Cannot peek item.")
80             return None
81
82     def is_empty(self):
83         return len(self.queue) == 0
84
85     def display(self):
86         if not self.is_empty():
87             print("Queue contents:")
88             for item in self.queue:
89                 print(item)
90         else:
91             print("Queue is empty.")
92
93 # Example usage
94 if __name__ == "__main__":
95     queue = Queue()
96     queue.enqueue(1)
97     queue.enqueue(2)
98     queue.enqueue(3)
99     queue.display()
100     queue.peek()
101     queue.dequeue()
102     queue.display()
103     print("Is queue empty?", queue.is_empty())
104     queue.dequeue()
105     queue.dequeue()
106     print("Is queue empty?", queue.is_empty())
```

Output:

```

Is stack empty: True
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Item '1' enqueued to queue.
Item '2' enqueued to queue.
Item '3' enqueued to queue.
Queue contents:
1
2
3
Item at front of queue: '1'
Item '1' dequeued from queue.
Queue contents:
2
3
Is queue empty? False
Item '2' dequeued from queue.
Item '3' dequeued from queue.
Is queue empty? True
PS C:\Users\arell\Music\aiac>

```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt:

#write a python code to implemwent to generate a Singly Linked List with insert and display methods

Code:

```

160 #write a python code to implement to generate a Singly Linked List with insert and display methods
161 class Node:
162     def __init__(self, data):
163         self.data = data
164         self.next = None
165 class SinglyLinkedList:
166     def __init__(self):
167         self.head = None
168
169     def insert(self, data):
170         new_node = Node(data)
171         if self.head is None:
172             self.head = new_node
173             print(f"Inserted head node with value: {data}")
174         else:
175             current = self.head
176             while current.next:
177                 current = current.next
178             current.next = new_node
179             print(f"Inserted node with value: {data} at the end of the list")
180
181     def display(self):
182         if self.head is None:
183             print("The linked list is empty.")
184         else:
185             print("Linked List contents:")
186             current = self.head
187             while current:
188                 print(current.data)
189                 current = current.next
190 # Example usage
191 if __name__ == "__main__":
192     linked_list = SinglyLinkedList()
193     linked_list.insert(1)
194     linked_list.insert(2)
195     linked_list.insert(3)
196     linked_list.display()
197

```

Output:

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Inserted head node with value: 1
Inserted node with value: 2 at the end of the list
Inserted node with value: 3 at the end of the list
Linked List contents:
1
2
3
PS C:\Users\arell\Music\aiac> 

```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

Promot:

#write a python code to implement the binary search tree with insert and in-order traversal methods.

Code:

```
107 #write a python code to implement the binary search tree with insert and in-order traversal methods.
108 class TreeNode:
109     def __init__(self, key):
110         self.left = None
111         self.right = None
112         self.val = key
113 class BinarySearchTree:
114     def __init__(self):
115         self.root = None
116
117     def insert(self, key):
118         if self.root is None:
119             self.root = TreeNode(key)
120             print(f"Inserted root node with value: {key}")
121         else:
122             self._insert_recursively(self.root, key)
123
124     def _insert_recursively(self, node, key):
125         if key < node.val:
126             if node.left is None:
127                 node.left = TreeNode(key)
128                 print(f"Inserted node with value: {key} to the left of {node.val}")
129             else:
130                 self._insert_recursively(node.left, key)
131         else:
132             if node.right is None:
133                 node.right = TreeNode(key)
134                 print(f"Inserted node with value: {key} to the right of {node.val}")
135             else:
136                 self._insert_recursively(node.right, key)
137
138     def in_order_traversal(self):
139         print("In-order traversal of the binary search tree:")
140         self._in_order_recursively(self.root)
141
142     def _in_order_recursively(self, node):
143         if node is not None:
144             self._in_order_recursively(node.left)
145             print(node.val)
146             self._in_order_recursively(node.right)
147
148 # Example usage
149 if __name__ == "__main__":
150     bst = BinarySearchTree()
151     bst.insert(5)
152     bst.insert(3)
153     bst.insert(7)
154     bst.insert(2)
155     bst.insert(4)
156     bst.insert(6)
157     bst.insert(8)
158     bst.in_order_traversal()
```

Output:

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Inserted root node with value: 5
Inserted node with value: 3 to the left of 5
Inserted node with value: 7 to the right of 5
Inserted node with value: 2 to the left of 3
Inserted node with value: 4 to the right of 3
Inserted node with value: 6 to the left of 7
Inserted node with value: 8 to the right of 7
In-order traversal of the binary search tree:
2
3
4
5
6
7
8
PS C:\Users\arell\Music\aiac>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

#write a python code to implement the hash table to implement a hash table with basic insert, search, and delete methods.

Code:

```

198 #write a python code to implement the hash table to implement a hash table with basic insert, search, and delete methods.
199 class HashTable:
200     def __init__(self):
201         self.table = {}
202
203     def insert(self, key, value):
204         self.table[key] = value
205         print(f"Inserted key: '{key}' with value: '{value}'")
206
207     def search(self, key):
208         if key in self.table:
209             print(f"Key: '{key}' found with value: '{self.table[key]}'")
210             return self.table[key]
211         else:
212             print(f"Key: '{key}' not found.")
213             return None
214
215     def delete(self, key):
216         if key in self.table:
217             del self.table[key]
218             print(f"Key: '{key}' deleted from hash table.")
219             return True
220         else:
221             print(f"Key: '{key}' not found. Cannot delete.")
222             return False
223
224 # Example usage
225 if __name__ == "__main__":
226     hash_table = HashTable()
227     hash_table.insert("name", "Alice")
228     hash_table.insert("age", 30)
229     hash_table.search("name")
230     hash_table.search("age")
231     hash_table.delete("name")
232     hash_table.search("name")
233     hash_table.search("age")

```

Output:

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Inserted key: 'name' with value: 'Alice'
Inserted key: 'age' with value: '30'
Key: 'name' found with value: 'Alice'
Key: 'age' found with value: '30'
Key: 'name' deleted from hash table.
Key: 'name' not found.
Key: 'age' found with value: '30'
PS C:\Users\arell\Music\aiac> 

```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt:

#write a python code to implement a graph using an adjacency list.

```

234 #write a python code to implement a graph using an adjacency list.
235 class Graph:
236     def __init__(self):
237         self.adjacency_list = {}
238
239     def add_edge(self, vertex1, vertex2):
240         if vertex1 not in self.adjacency_list:
241             self.adjacency_list[vertex1] = []
242         if vertex2 not in self.adjacency_list:
243             self.adjacency_list[vertex2] = []
244         self.adjacency_list[vertex1].append(vertex2)
245         self.adjacency_list[vertex2].append(vertex1)
246         print(f"Edge added between '{vertex1}' and '{vertex2}'")
247
248     def display(self):
249         print("Graph adjacency list:")
250         for vertex, neighbors in self.adjacency_list.items():
251             print(f"{vertex}: {' '.join(neighbors)}")
252 # Example usage
253 if __name__ == "__main__":
254     graph = Graph()
255     graph.add_edge("A", "B")
256     graph.add_edge("A", "C")
257     graph.add_edge("B", "D")
258     graph.add_edge("C", "D")
259     graph.display()
260
261

```

Output:

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Edge added between 'A' and 'B'
Edge added between 'A' and 'C'
Edge added between 'B' and 'D'
Edge added between 'C' and 'D'
Graph adjacency list:
A: B, C
B: A, D
C: A, D
D: B, C
PS C:\Users\arell\Music\aiac>

```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority),

and display methods.

Prompt:

#write a python code to implement a priority queue using Python's heapq module.

Code:

```
263 #write a python code to implement a priority queue using Python's heapq module.
264 import heapq
265 class PriorityQueue:
266     def __init__(self):
267         self.elements = []
268
269     def push(self, item, priority):
270         heapq.heappush(self.elements, (priority, item))
271         print(f"Item '{item}' with priority '{priority}' added to the priority queue.")
272
273     def pop(self):
274         if not self.is_empty():
275             priority, item = heapq.heappop(self.elements)
276             print(f"Item '{item}' with priority '{priority}' popped from the priority queue.")
277             return item
278         else:
279             print("Priority queue is empty. Cannot pop item.")
280             return None
281
282     def peek(self):
283         if not self.is_empty():
284             priority, item = self.elements[0]
285             print(f"Item at top of priority queue: '{item}' with priority '{priority}'")
286             return item
287         else:
288             print("Priority queue is empty. Cannot peek item.")
289             return None
290
291     def is_empty(self):
292         return len(self.elements) == 0
293
294     def display(self):
295         if not self.is_empty():
296             print("Priority Queue contents:")
297             for priority, item in sorted(self.elements):
298                 print(f"Item: '{item}', Priority: '{priority}'")
299         else:
300             print("Priority queue is empty.")
301
302 # Example usage
303 if __name__ == "__main__":
304     pq = PriorityQueue()
305     pq.push("Task 1", 2)
306     pq.push("Task 2", 1)
307     pq.push("Task 3", 3)
308     pq.display()
309     pq.peek()
310     pq.pop()
311     pq.display()
312     print(f"Is priority queue empty?", pq.is_empty())
313     pq.pop()
314     pq.pop()
315     print(f"Is priority queue empty?", pq.is_empty())
```

Output:

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
Item 'Task 1' with priority '2' added to the priority queue.
Item 'Task 2' with priority '1' added to the priority queue.
Item 'Task 3' with priority '3' added to the priority queue.
Priority Queue contents:
Item: 'Task 2', Priority: '1'
Item: 'Task 1', Priority: '2'
Item: 'Task 3', Priority: '3'
Item at top of priority queue: 'Task 2' with priority '1'
Item 'Task 2' with priority '1' popped from the priority queue.
Priority Queue contents:
Item: 'Task 1', Priority: '2'
Item: 'Task 3', Priority: '3'
Is priority queue empty? False
Item 'Task 1' with priority '2' popped from the priority queue.
Item 'Task 3' with priority '3' popped from the priority queue.
Is priority queue empty? True
PS C:\Users\arell\Music\aiac> 

```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using
collections.deque.

Sample Input Code:

```

class DequeDS:
    pass

```

Expected Output:

- Insert and remove from both ends with docstrings.

Prompt:

#write a python code to implement a double-ended queue using collections.deque.

Code:

```

316 #write a python code to implement a double-ended queue using collections.deque.
317 from collections import deque
318 class DoubleEndedQueue:
319     def __init__(self):
320         self.deque = deque()
321
322     def add_front(self, item):
323         self.deque.appendleft(item)
324         print(f"Item '{item}' added to the front of the deque.")
325
326     def add_rear(self, item):
327         self.deque.append(item)
328         print(f"Item '{item}' added to the rear of the deque.")
329
330     def remove_front(self):
331         if not self.is_empty():
332             item = self.deque.popleft()
333             print(f"Item '{item}' removed from the front of the deque.")
334             return item
335         else:
336             print("Deque is empty. Cannot remove item from front.")
337             return None
338
339     def remove_rear(self):
340         if not self.is_empty():
341             item = self.deque.pop()
342             print(f"Item '{item}' removed from the rear of the deque.")
343             return item
344         else:
345             print("Deque is empty. Cannot remove item from rear.")
346             return None
347
348     def peek_front(self):
349         if not self.is_empty():
350             item = self.deque[0]
351             print(f"Item at front of deque: '{item}'")
352             return item
353         else:
354             print("Deque is empty. Cannot peek front item.")
355             return None
356
357     def peek_rear(self):
358         if not self.is_empty():
359             item = self.deque[-1]
360             print(f"Item at rear of deque: '{item}'")
361             return item
362         else:
363             print("Deque is empty. Cannot peek rear item.")
364             return None
365
366     def is_empty(self):
367         return len(self.deque) == 0
368
369     def display(self):
370         if not self.is_empty():
371             print("Deque contents:")
372             for item in self.deque:
373                 print(item)
374         else:
375             print("Deque is empty.")
376
377 # Example usage
378 if __name__ == "__main__":
379     deq = DoubleEndedQueue()
380     deq.add_rear(1)
381     deq.add_rear(2)
382     deq.add_front(0)
383     deq.display()
384     deq.peek_front()
385     deq.peek_rear()
386     deq.remove_front()
387     deq.remove_rear()
388     deq.display()
389     print("Is deque empty?", deq.is_empty())
390     deq.remove_front()
391     print("Is deque empty?", deq.is_empty())
392

```

Output:

```
Item '1' added to the rear of the deque.
Item '2' added to the rear of the deque.
Item '0' added to the front of the deque.
Deque contents:
0
1
2
Item at front of deque: '0'
Item at rear of deque: '2'
Item '0' removed from the front of the deque.
Item '2' removed from the rear of the deque.
Deque contents:
1
Is deque empty? False
Item '1' removed from the front of the deque.
Is deque empty? True
PS C:\Users\arell\Music\aiac> 
```

Task Description #9 Real-Time Application Challenge – Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Task Description #10: Smart Hospital Management System – Data

Code:

```

421
422 # Campus Resource Management System
423 # Mapping Features to Data Structures with Justifications
424
425 """
426 FEATURE MAPPING TABLE:
427 =====
428 1. Student Attendance Tracking → Linked List
429    Justification: Maintains chronological order of entry/exit logs. Easy to insert at end and traverse
430    sequentially for daily reports without needing random access.
431
432 2. Event Registration System → Hash Table
433    Justification: Provides O(1) average lookup, insertion, and deletion of participants by ID. Enables
434    quick search and removal operations for managing registrations efficiently.
435
436 3. Library Book Borrowing → BST
437    Justification: Books can be organized by ISBN or title for efficient searching. In-order traversal
438    provides sorted book catalog, and BST allows quick insertion/deletion of borrowed/returned books.
439
440 4. Bus Scheduling System → Graph
441    Justification: Represents bus routes as vertices and connections between stops as edges. Models
442    complex route networks and enables pathfinding between different campus stops.
443
444 5. Cafeteria Order Queue → Queue (FIFO)
445    Justification: Students must be served in arrival order. Queue's FIFO nature ensures fairness and
446    efficient management of student orders without priority bias.
447 """
448
449 # IMPLEMENTATION: Library Book Borrowing System using Binary Search Tree
450
451 class BookNode:
452     """Node class for BST storing book information"""
453     def __init__(self, isbn, title, author):
454         self.isbn = isbn
455         self.title = title
456         self.author = author
457         self.left = None
458         self.right = None
459         self.is_available = True
460
461 class LibraryBookSystem:
462     """Binary Search Tree implementation for managing library books"""
463     def __init__(self):
464         self.root = None
465
466     def insert_book(self, isbn, title, author):
467         """Insert a new book into the library system"""
468         if self.root is None:
469             self.root = BookNode(isbn, title, author)
470             print(f"📖 Added Book: '{title}' (ISBN: {isbn})")
471         else:
472             self._insert_recursively(self.root, isbn, title, author)
473
474     def _insert_recursively(self, node, isbn, title, author):
475         """Helper method to insert book recursively"""
476         if isbn < node.isbn:
477             if node.left is None:
478                 node.left = BookNode(isbn, title, author)
479                 print(f"📖 Added Book: '{title}' (ISBN: {isbn})")
480             else:
481                 self._insert_recursively(node.left, isbn, title, author)
482         else:
483             if node.right is None:
484                 node.right = BookNode(isbn, title, author)
485                 print(f"📖 Added Book: '{title}' (ISBN: {isbn})")
486             else:
487                 self._insert_recursively(node.right, isbn, title, author)
488
489     def search_book(self, isbn):
490         """Search for a book by ISBN"""
491         result = self._search_recursively(self.root, isbn)
492         if result:
493             status = "✅ Available" if result.is_available else "❌ Not Available"
494             print(f"Found: '{result.title}' by {result.author} - {status}")
495             return result
496         else:
497             print(f"Book with ISBN {isbn} not found.")
498             return None
499
500     def _search_recursively(self, node, isbn):
501         """Helper method to search recursively"""
502         if node is None:
503             return None
504         if isbn == node.isbn:
505             return node
506         elif isbn < node.isbn:
507             return self._search_recursively(node.left, isbn)
508         else:
509             return self._search_recursively(node.right, isbn)
510
511     def borrow_book(self, isbn):
512         """Mark a book as borrowed"""
513         book = self.search_book(isbn)
514         if book and book.is_available:
515             book.is_available = False
516             print(f"📖✅ Book borrowed: '{book.title}'")
517         elif book:
518             print(f"❌ Book '{book.title}' is already borrowed.")
519
520

```

```

520
521     def return_book(self, isbn):
522         """Mark a book as returned"""
523         book = self.search_book(isbn)
524         if book and not book.is_available:
525             book.is_available = True
526             print(f"👍 Book returned: '{book.title}'")
527         elif book:
528             print(f"⚠️ Book '{book.title}' was not borrowed.")
529
530     def display_all_books(self):
531         """Display all books in sorted order (in-order traversal)"""
532         if self.root is None:
533             print("📖 Library is empty.")
534         else:
535             print("\n📖 Library Catalog (Sorted by ISBN):")
536             print("-" * 60)
537             self._in_order_traversal(self.root)
538             print("-" * 60 + "\n")
539
540     def _in_order_traversal(self, node):
541         """In-order traversal to display books in sorted order"""
542         if node is not None:
543             self._in_order_traversal(node.left)
544             status = "Available" if node.is_available else "Borrowed"
545             print(f"ISBN: {node.isbn} | {node.title} | {node.author} | {status}")
546             self._in_order_traversal(node.right)
547
548
549 # Example Usage
550 if __name__ == "__main__":
551     print("-" * 60)
552     print("🏫 CAMPUS LIBRARY BOOK MANAGEMENT SYSTEM")
553     print("-" * 60 + "\n")
554
555     library = LibraryBookSystem()
556
557     # Add books
558     library.insert_book(101, "Python Programming", "Guido van Rossum")
559     library.insert_book(105, "Data Structures", "Mark Allen Weiss")
560     library.insert_book(98, "Algorithms", "Thomas Cormen")
561     library.insert_book(110, "Web Development", "Kyle Simpson")
562     library.insert_book(95, "Database Design", "C.J. Date")
563
564     # Display all books
565     library.display_all_books()
566
567     # Borrow books
568     print("📖 Borrowing Operations:")
569     library.borrow_book(101)
570     library.borrow_book(105)
571
572     # Try to borrow already borrowed book
573     library.borrow_book(101)
574
575     # Display updated catalog
576     library.display_all_books()
577
578     # Search for a book
579     print("🔍 Search Operations:")
580     library.search_book(98)
581     library.search_book(999)
582
583     # Return books
584     print("\n📖 Returning Operations:")
585     library.return_book(101)
586     library.return_book(105)
587
588     # Final catalog
589     library.display_all_books()
590
591

```

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
```

CAMPUS LIBRARY BOOK MANAGEMENT SYSTEM

Added Book: 'Python Programming' (ISBN: 101)
Added Book: 'Data Structures' (ISBN: 105)
Added Book: 'Algorithms' (ISBN: 98)
Added Book: 'Web Development' (ISBN: 110)
Added Book: 'Database Design' (ISBN: 95)

Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

Borrowing Operations:

Found: 'Python Programming' by Guido van Rossum - Available
Book borrowed: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - Available
Book borrowed: 'Data Structures'
Found: 'Python Programming' by Guido van Rossum - Not Available
Book 'Python Programming' is already borrowed.

Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Borrowed
ISBN: 105 | Data Structures | Mark Allen Weiss | Borrowed
ISBN: 110 | Web Development | Kyle Simpson | Available

Search Operations:

Found: 'Algorithms' by Thomas Cormen - Available
Book with ISBN 999 not found.

Returning Operations:

Found: 'Python Programming' by Guido van Rossum - Not Available
Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - Not Available
Book returned: 'Data Structures'

Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

ISBN: 105 | Data Structures | Mark Allen Weiss | Borrowed
ISBN: 110 | Web Development | Kyle Simpson | Available

Search Operations:

Found: 'Algorithms' by Thomas Cormen - Available
Book with ISBN 999 not found.

Returning Operations:

Found: 'Python Programming' by Guido van Rossum - Not Available
Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - Not Available
Book returned: 'Data Structures'

Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

ISBN: 110 | Web Development | Kyle Simpson | Available

🔍 Search Operations:

Found: 'Algorithms' by Thomas Cormen - ✅ Available
Book with ISBN 999 not found.

📚 Returning Operations:

Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
✅ Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ❌ Not Available
✅ Book returned: 'Data Structures'

📖 Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

Found: 'Algorithms' by Thomas Cormen - ✅ Available
Book with ISBN 999 not found.

📚 Returning Operations:

Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
✅ Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ❌ Not Available
✅ Book returned: 'Data Structures'

📖 Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

📚 Returning Operations:

Found: 'Python Programming' by Guido van Rossum - ❌ Not Available
✅ Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ❌ Not Available
✅ Book returned: 'Data Structures'

📖 Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

✅ Book returned: 'Python Programming'
Found: 'Data Structures' by Mark Allen Weiss - ❌ Not Available
✅ Book returned: 'Data Structures'

📖 Library Catalog (Sorted by ISBN):

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

ISBN: 95 | Database Design | C.J. Date | Available
ISBN: 98 | Algorithms | Thomas Cormen | Available
ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

ISBN: 101 | Python Programming | Guido van Rossum | Available
ISBN: 105 | Data Structures | Mark Allen Weiss | Available
ISBN: 110 | Web Development | Kyle Simpson | Available

Task Description #10: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

586 """
587
588 FEATURE MAPPING TABLE:
589 =====
590 1. Patient Check-In System → Queue
591 | Justification: Patients are treated in FIFO order based on arrival time. Queue ensures fair
592 | service and maintains chronological order for non-emergency patients.
593
594 2. Emergency Case Handling → Priority Queue
595 | Justification: Critical patients must be prioritized regardless of arrival time. Priority Queue
596 | allows sorting by severity level, ensuring urgent cases are treated first.
597
598 3. Medical Records Storage → Hash Table
599 | Justification: Provides O(1) average lookup time for patient records using patient ID as key.
600 | Enables fast retrieval and updating of medical information.
601
602 4. Doctor Appointment Scheduling → Binary Search Tree (BST)
603 | Justification: Appointments can be organized by time slots for efficient searching and scheduling.
604 | In-order traversal provides sorted appointment schedule.
605
606 5. Hospital Room Navigation → Graph
607 | Justification: Represents hospital layout with wards and rooms as vertices and connections as edges.
608 | Enables pathfinding and navigation between different hospital sections.
609 """
610
611 # IMPLEMENTATION: Emergency Case Handling using Priority Queue
612 import heapq
613
614 class Patient:
615     def __init__(self, patient_id, name, severity, arrival_time):
616         self.patient_id = patient_id
617         self.name = name
618         self.severity = severity # 1=Critical, 2=High, 3=Medium, 4=Low
619         self.arrival_time = arrival_time
620         self.status = "Waiting"
621
622     def __lt__(self, other):
623         return self.severity < other.severity
624
625     def __repr__(self):
626         return f"Patient({self.patient_id}, {self.name}, Severity: {self.severity})"
627
628
629 class EmergencyManagementSystem:
630     """Manages emergency patient queue using priority queue."""
631
632     def __init__(self):
633         self.emergency_queue = []
634         self.treated_patients = []
635
636     def add_patient(self, patient_id, name, severity, arrival_time):
637         """Add a patient to the emergency queue."""
638         patient = Patient(patient_id, name, severity, arrival_time)
639         heapq.heappush(self.emergency_queue, patient)
640         severity_map = {1: "🔴 CRITICAL", 2: "🟠 HIGH", 3: "🟡 MEDIUM", 4: "🟢 LOW"}
641         print(f"✅ Patient Added: {name} (ID: {patient_id}) - {severity_map[severity]}")
642
643     def treat_patient(self):
644         """Treat the next patient with highest priority."""
645         if not self.is_empty():
646             patient = heapq.heappop(self.emergency_queue)
647             patient.status = "Treated"
648             self.treated_patients.append(patient)
649             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
650             print(f"🏥 Now Treating: {patient.name} (Severity: {severity_map[patient.severity]})")
651             return patient
652         else:
653             print("❌ No patients in queue.")
654             return None
655

```

```

629 class EmergencyManagementSystem:
630
631     def peek_next_patient(self):
632         """View next patient to be treated."""
633         if not self.is_empty():
634             patient = self.emergency_queue[0]
635             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
636             print(f"🏥 Next Patient: {patient.name} (Severity: {severity_map[patient.severity]})")
637             return patient
638         else:
639             print("❌ No patients in queue.")
640             return None
641
642     def is_empty(self):
643         """Check if queue is empty."""
644         return len(self.emergency_queue) == 0
645
646     def display_queue(self):
647         """Display all patients waiting in queue."""
648         if self.is_empty():
649             print("🏥 No patients waiting.\n")
650         else:
651             print("\n🏥 EMERGENCY QUEUE (Sorted by Priority):")
652             print("-" * 70)
653             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
654             for patient in sorted(self.emergency_queue):
655                 print(f"ID: {patient.patient_id} | Name: {patient.name} | Severity: {severity_map[patient.severity]} | Status: {patient.status}")
656             print("-" * 70 + "\n")
657
658     def display_treated_patients(self):
659         """Display all treated patients."""
660         if not self.treated_patients:
661             print("🏥 No treated patients yet.\n")
662         else:
663             print("\n✅ TREATED PATIENTS:")
664             print("-" * 70)
665             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
666             for patient in self.treated_patients:
667                 print(f"ID: {patient.patient_id} | Name: {patient.name} | Severity: {severity_map[patient.severity]}")
668             print("-" * 70 + "\n")
669
670 # Example Usage
671 if __name__ == "__main__":
672     print("-" * 70)
673     print("🏥 SMART HOSPITAL EMERGENCY MANAGEMENT SYSTEM")
674     print("-" * 70 + "\n")
675
676     hospital = EmergencyManagementSystem()
677
678     # Add patients with different severity levels
679     print("📋 Adding Patients to Emergency Queue:\n")
680     hospital.add_patient(1, "John Doe", 4, "09:00")
681     hospital.add_patient(2, "Sarah Smith", 1, "09:05")
682     hospital.add_patient(3, "Mike Johnson", 3, "09:10")
683     hospital.add_patient(4, "Emily Brown", 2, "09:15")
684     hospital.add_patient(5, "David Wilson", 1, "09:20")
685
686     # Display queue
687     hospital.display_queue()
688
689     # Peek next patient
690     print("🏥 Checking Next Patient:\n")
691     hospital.peek_next_patient()
692
693     # Treat patients
694     print("\n🏥 TREATMENT PROCESS:\n")
695     hospital.treat_patient()
696     hospital.treat_patient()
697     hospital.treat_patient()
698
699     # Display remaining queue
700     hospital.display_queue()
701
702     # Display treated patients
703     hospital.display_treated_patients()
704
705     # Treat remaining patients
706     print("Treating remaining patients:\n")

```

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
```

```
=====
SMART HOSPITAL EMERGENCY MANAGEMENT SYSTEM
=====
```

```
📋 Adding Patients to Emergency Queue:
```

```
✅ Patient Added: John Doe (ID: 1) - 🟢 LOW
✅ Patient Added: Sarah Smith (ID: 2) - 🔴 CRITICAL
✅ Patient Added: Mike Johnson (ID: 3) - 🟡 MEDIUM
✅ Patient Added: Emily Brown (ID: 4) - 🟠 HIGH
✅ Patient Added: David Wilson (ID: 5) - 🔴 CRITICAL
```

```
🚑 EMERGENCY QUEUE (Sorted by Priority):
```

```
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL | Status: Waiting
ID: 5 | Name: David Wilson | Severity: CRITICAL | Status: Waiting
ID: 4 | Name: Emily Brown | Severity: HIGH | Status: Waiting
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Status: Waiting
ID: 1 | Name: John Doe | Severity: LOW | Status: Waiting
-----
```

```
🔍 Checking Next Patient:
```

```
📋 Next Patient: Sarah Smith (Severity: CRITICAL)
```

```
📋 TREATMENT PROCESS:
```

```
📋 Now Treating: Sarah Smith (Severity: CRITICAL)
📋 Now Treating: David Wilson (Severity: CRITICAL)
📋 Now Treating: Emily Brown (Severity: HIGH)
```

```
🚑 EMERGENCY QUEUE (Sorted by Priority):
```

```
-----
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Status: Waiting
ID: 1 | Name: John Doe | Severity: LOW | Status: Waiting
-----
```

```
✅ TREATED PATIENTS:
```

```
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
-----
```

```
Treating remaining patients:
```

```
📋 Now Treating: Mike Johnson (Severity: MEDIUM)
📋 Now Treating: John Doe (Severity: LOW)
```

```
✅ TREATED PATIENTS:
```

```
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
ID: 3 | Name: Mike Johnson | Severity: MEDIUM
ID: 1 | Name: John Doe | Severity: LOW
-----
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected

logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

733
734 """Smart City Traffic Control System.
735
736 This module:
737 1) maps each required feature to the most suitable data structure, and
738 2) implements Emergency Vehicle Priority Handling using a Priority Queue.
739
740 Priority scale used in implementation:
741 1 = Critical, 2 = High, 3 = Medium, 4 = Low.
742 """
743
744 from typing import List, Tuple, Optional
745 from dataclasses import dataclass, field
746 from itertools import count
747 import heapq
748
749
750 # Feature -> Data Structure -> Justification (2 sentences each)
751 FEATURE_MAPPING: List[Tuple[str, str, str]] = [
752     (
753         "Traffic Signal Queue",
754         "Queue",
755         "Vehicles at a signal are processed in the same order they arrive. "
756         "A queue enforces FIFO behavior and keeps flow fair."
757     ),
758     (
759         "Emergency Vehicle Priority Handling",
760         "Priority Queue",
761         "Ambulances and fire trucks must move before normal vehicles. "
762         "A priority queue always serves the highest-priority request first."
763     ),
764     (
765         "Vehicle Registration Lookup",
766         "Hash Table",
767         "Vehicle details are usually searched by registration number. "
768         "A hash table gives average O(1) lookup, insert, and delete."
769     ),
770     (
771         "Road Network Mapping",
772         "Graph",
773         "Intersections and roads are naturally represented as nodes and edges. "
774         "A graph supports connectivity checks and route finding."
775     ),
776     (
777         "Parking Slot Availability",
778         "Hash Table",
779         "Each slot ID can map directly to occupied/available status. "
780         "A hash table allows quick status checks and updates."
781     ),
782 ]
783
784
785 def print_feature_mapping_table() -> None:
786     """Print the feature mapping table."""
787     print("\nSMART CITY TRAFFIC CONTROL - FEATURE MAPPING")
788     print("=" * 120)
789     print(f'{"Feature":<36} {"Data Structure":<18} Justification')
790     print("-" * 120)
791     for feature, structure, reason in FEATURE_MAPPING:
792         print(f'{"feature":<36} {"structure":<18} {"reason}")
793     print("=" * 120)
794
795

```

```

796 @dataclass(order=True)
797 class EmergencyVehicle:
798     """Store one emergency vehicle request for priority dispatch."""
799     sequence: int
800     vehicle_id: str = field(compare=False)
801     vehicle_type: str = field(compare=False)
802     location: str = field(compare=False)
803
804
805
806 class EmergencyVehiclePrioritySystem:
807     """Priority-based dispatcher for emergency vehicles."""
808
809     def __init__(self) -> None:
810         """Initialize an empty emergency priority queue."""
811         self._queue: List[EmergencyVehicle] = []
812         self._counter = count()
813
814     @staticmethod
815     def _label(priority: int) -> str:
816         """Convert numeric priority to text."""
817         return {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}.get(priority, "UNKNOWN")
818
819     def add_vehicle(self, vehicle_id: str, vehicle_type: str, location: str, priority: int) -> None:
820         """Add an emergency vehicle request."""
821         req = EmergencyVehicle(
822             priority=priority,
823             sequence=next(self._counter),
824             vehicle_id=vehicle_id,
825             vehicle_type=vehicle_type,
826             location=location,
827         )
828         heapq.heappush(self._queue, req)
829         print(f"Added: {req.vehicle_type} ({req.vehicle_id}) at {req.location} [{self._label(priority)}]")
830
831     def dispatch_next(self) -> Optional[EmergencyVehicle]:
832         """Dispatch and return the highest-priority vehicle."""
833         if not self._queue:
834             print("No emergency vehicles waiting.")
835             return None
836         req = heapq.heappop(self._queue)
837         print(f"Dispatching: {req.vehicle_type} ({req.vehicle_id}) from {req.location} [{self._label(req.priority)}]")
838         return req
839
840     def peek_next(self) -> Optional[EmergencyVehicle]:
841         """Show the next vehicle to dispatch without removing it."""
842         if not self._queue:
843             print("No emergency vehicles waiting.")
844             return None
845         req = self._queue[0]
846         print(f"Next: {req.vehicle_type} ({req.vehicle_id}) at {req.location} [{self._label(req.priority)}]")
847         return req
848
849     def is_empty(self) -> bool:
850         """Return True if no vehicles are waiting."""
851         return len(self._queue) == 0
852
853     def display_queue(self) -> None:
854         """Display waiting vehicles in dispatch order."""
855         if not self._queue:
856             print("Queue is empty.")
857         return

```

```

858
859     print("\nEmergency Queue (highest priority first):")
860     print("-" * 80)
861     for req in sorted(self._queue):
862         print(
863             f"ID: {req.vehicle_id:<8} Type: {req.vehicle_type:<12} "
864             f"Location: {req.location:<15} Priority: {self._label(req.priority)}"
865         )
866     print("-" * 80)
867
868
869 if __name__ == "__main__":
870     # Print required mapping table
871     print_feature_mapping_table()
872
873     # Implemented feature demo: Emergency Vehicle Priority Handling
874     system = EmergencyVehiclePrioritySystem()
875
876     print("\nAdding emergency vehicles:")
877     system.add_vehicle("AMB-101", "Ambulance", "Junction A", 1)
878     system.add_vehicle("FIRE-007", "Fire Truck", "Junction C", 2)
879     system.add_vehicle("POL-055", "Police Van", "Junction B", 3)
880     system.add_vehicle("AMB-212", "Ambulance", "Junction D", 1)
881
882     system.display_queue()
883     system.peak_next()
884
885     print("\nDispatch process:")
886     while not system.is_empty():
887         system.dispatch_next()

```

Output:

```

Vehicle Registration Lookup: Hash Table
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"

SMART CITY TRAFFIC CONTROL - FEATURE MAPPING
=====
Feature                Data Structure    Justification
-----
Traffic Signal Queue    Queue             Vehicles at a signal are processed in the same order they arrive. A queue enforces FIFO behavior and keeps flow fair.
Emergency Vehicle Priority Handling Priority Queue      Ambulances and fire trucks must move before normal vehicles. A priority queue always serves the highest-priority request first.
Vehicle Registration Lookup Hash Table         Vehicle details are usually searched by registration number. A hash table gives average O(1) lookup, insert, and delete.
Road Network Mapping    Graph             Intersections and roads are naturally represented as nodes and edges. A graph supports connectivity checks and route finding.
Parking Slot Availability Hash Table         Each slot ID can map directly to occupied/available status. A hash table allows quick status checks and updates.
=====
Feature                Data Structure    Justification
-----
Traffic Signal Queue    Queue             Vehicles at a signal are processed in the same order they arrive. A queue enforces FIFO behavior and keeps flow fair.
Emergency Vehicle Priority Handling Priority Queue      Ambulances and fire trucks must move before normal vehicles. A priority queue always serves the highest-priority request first.
Vehicle Registration Lookup Hash Table         Vehicle details are usually searched by registration number. A hash table gives average O(1) lookup, insert, and delete.
Road Network Mapping    Graph             Intersections and roads are naturally represented as nodes and edges. A graph supports connectivity checks and route finding.
Parking Slot Availability Hash Table         Each slot ID can map directly to occupied/available status. A hash table allows quick status checks and updates.
=====

Adding emergency vehicles:
Added: Ambulance (AMB-101) at Junction A [CRITICAL]
Added: Fire Truck (FIRE-007) at Junction C [HIGH]
Added: Police Van (POL-055) at Junction B [MEDIUM]
Vehicle Registration Lookup Hash Table         Vehicle details are usually searched by registration number. A hash table gives average O(1) lookup, insert, and delete.
Road Network Mapping    Graph             Intersections and roads are naturally represented as nodes and edges. A graph supports connectivity checks and route finding.
Parking Slot Availability Hash Table         Each slot ID can map directly to occupied/available status. A hash table allows quick status checks and updates.
=====

Adding emergency vehicles:
Added: Ambulance (AMB-101) at Junction A [CRITICAL]
Added: Fire Truck (FIRE-007) at Junction C [HIGH]
Added: Police Van (POL-055) at Junction B [MEDIUM]
Added: Ambulance (AMB-212) at Junction D [CRITICAL]

=====

Adding emergency vehicles:
Added: Ambulance (AMB-101) at Junction A [CRITICAL]
Added: Fire Truck (FIRE-007) at Junction C [HIGH]
Added: Police Van (POL-055) at Junction B [MEDIUM]
Added: Ambulance (AMB-212) at Junction D [CRITICAL]

=====

Emergency Queue (highest priority first):
Added: Ambulance (AMB-212) at Junction D [CRITICAL]

Emergency Queue (highest priority first):
ID: AMB-101 Type: Ambulance Location: Junction A Priority: CRITICAL
ID: AMB-212 Type: Ambulance Location: Junction D Priority: CRITICAL
Emergency Queue (highest priority first):
ID: AMB-101 Type: Ambulance Location: Junction A Priority: CRITICAL
ID: AMB-212 Type: Ambulance Location: Junction D Priority: CRITICAL
ID: FIRE-007 Type: Fire Truck Location: Junction C Priority: HIGH
ID: AMB-101 Type: Ambulance Location: Junction A Priority: CRITICAL
ID: AMB-212 Type: Ambulance Location: Junction D Priority: CRITICAL
ID: FIRE-007 Type: Fire Truck Location: Junction C Priority: HIGH
ID: POL-055 Type: Police Van Location: Junction B Priority: MEDIUM

Next: Ambulance (AMB-101) at Junction A [CRITICAL]
ID: FIRE-007 Type: Fire Truck Location: Junction C Priority: HIGH
ID: POL-055 Type: Police Van Location: Junction B Priority: MEDIUM

Next: Ambulance (AMB-101) at Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]
ID: POL-055 Type: Police Van Location: Junction B Priority: MEDIUM

Next: Ambulance (AMB-101) at Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]
Dispatching: Ambulance (AMB-212) from Junction D [CRITICAL]
Dispatching: Ambulance (AMB-212) from Junction D [CRITICAL]
Dispatching: Fire Truck (FIRE-007) from Junction C [HIGH]
Dispatching: Fire Truck (FIRE-007) from Junction C [HIGH]
Dispatching: Police Van (POL-055) from Junction B [MEDIUM]
PS C:\Users\arell\Music\aiac> 

```

```

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]
ID: POL-055 Type: Police Van Location: Junction B Priority: MEDIUM

Next: Ambulance (AMB-101) at Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]

Dispatch process:
Dispatching: Ambulance (AMB-101) from Junction A [CRITICAL]
Dispatching: Ambulance (AMB-212) from Junction D [CRITICAL]
Dispatching: Ambulance (AMB-212) from Junction D [CRITICAL]
Dispatching: Fire Truck (FIRE-007) from Junction C [HIGH]
Dispatching: Fire Truck (FIRE-007) from Junction C [HIGH]
Dispatching: Police Van (POL-055) from Junction B [MEDIUM]
PS C:\Users\arell\Music\aiac> 

```

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

879
880
881     Smart E-Commerce Platform 📦 Data Structure Challenge
882     An e-commerce company wants to build a Smart Online Shopping System
883     with:
884     1. Shopping Cart Management 📦 Add and remove products dynamically.
885     2. Order Processing System 📦 Orders processed in the order they are placed.
886     3. Top-Selling Products Tracker 📦 Products ranked by sales count.
887     4. Product Search Engine 📦 Fast lookup of products using product ID.
888     5. Delivery Route Planning 📦 Connect warehouses and delivery locations.
889
890
891
892     FEATURE MAPPING TABLE:
893     =====
894     1. Shopping Cart Management → Deque
895         Justification: Allows adding/removing products from both ends efficiently.
896         Supports undo (pop from end) and redo operations commonly needed in shopping carts.
897
898     2. Order Processing System → Queue
899         Justification: Orders must be processed in FIFO order based on arrival time.
900         Queue ensures fair processing and maintains chronological order for order fulfillment.
901
902     3. Top-Selling Products Tracker → Priority Queue
903         Justification: Products are ranked by sales count with highest-selling items prioritized.
904         Priority queue automatically maintains sorted order by sales frequency for quick access to top products.
905
906     4. Product Search Engine → Hash Table
907         Justification: Provides O(1) average lookup time for products using product ID as key.
908         Enables fast retrieval of product details like price, stock, and specifications.
909
910     5. Delivery Route Planning → Graph
911         Justification: Represents warehouses and delivery locations as vertices with roads as edges.
912         Graph enables pathfinding algorithms to optimize delivery routes and reduce shipping time.
913
914
915 # IMPLEMENTATION: Shopping Cart Management using Deque
916
917
918 from collections import deque
919
920
921 class Product:
922     def __init__(self, product_id, name, price, quantity):
923         self.product_id = product_id
924         self.name = name
925         self.price = price
926         self.quantity = quantity
927
928     def __repr__(self):
929         return f"Product({self.product_id}, {self.name}, ${self.price}, Qty: {self.quantity})"
930
931
932 class ShoppingCart:
933     """Shopping Cart implementation using Deque for efficient add/remove operations."""
934
935     def __init__(self):
936         self.cart = deque()
937
938     def add_product(self, product):
939         """Add a product to the end of the cart."""
940         self.cart.append(product)
941         print(f"✅ Added to cart: {product.name} - ${product.price}")
942

```

```

942
943 def add_product_front(self, product):
944     """Add a product to the front of the cart (priority addition)."""
945     self.cart.appendleft(product)
946     print(f"★ Priority added to cart: {product.name} - ${product.price}")
947
948 def remove_product(self):
949     """Remove the most recently added product (undo operation)."""
950     if not self.is_empty():
951         product = self.cart.pop()
952         print(f"✗ Removed from cart: {product.name}")
953         return product
954     else:
955         print("⚠ Cart is empty. Cannot remove.")
956         return None
957
958 def remove_product_front(self):
959     """Remove the first product from cart."""
960     if not self.is_empty():
961         product = self.cart.popleft()
962         print(f"✗ Removed from front: {product.name}")
963         return product
964     else:
965         print("⚠ Cart is empty. Cannot remove.")
966         return None
967
968 def get_total_price(self):
969     """Calculate total price of all items in cart."""
970     total = sum(product.price * product.quantity for product in self.cart)
971     return total
972
973 def is_empty(self):
974     """Check if cart is empty."""
975     return len(self.cart) == 0
976
977 def display_cart(self):
978     """Display all items in the cart."""
979     if self.is_empty():
980         print("🛒 Your cart is empty.\n")
981     else:
982         print("\n🛒 SHOPPING CART:")
983         print("-" * 70)
984         total = 0
985         for i, product in enumerate(self.cart, 1):
986             item_total = product.price * product.quantity
987             total += item_total
988             print(f"{i}. {product.name} | Price: ${product.price} | Qty: {product.quantity} | Subtotal: ${item_total}")
989         print("-" * 70)
990         print(f"Total Price: ${total}")
991         print("-" * 70 + "\n")
992

```

```

993
994 # Example Usage
995 if __name__ == "__main__":
996     print("=" * 70)
997     print("🛒 SMART E-COMMERCE SHOPPING CART SYSTEM")
998     print("=" * 70 + "\n")
999
1000     cart = ShoppingCart()
1001
1002     # Create products
1003     p1 = Product(101, "Laptop", 999.99, 1)
1004     p2 = Product(102, "Wireless Mouse", 29.99, 2)
1005     p3 = Product(103, "USB-C Cable", 15.99, 3)
1006     p4 = Product(104, "Monitor", 299.99, 1)
1007     p5 = Product(105, "Keyboard", 79.99, 1)
1008
1009     # Add products to cart
1010     print("🛒 Adding products to cart:\n")
1011     cart.add_product(p1)
1012     cart.add_product(p2)
1013     cart.add_product(p3)
1014     cart.add_product_front(p4) # Priority item
1015     cart.add_product(p5)
1016
1017     # Display cart
1018     cart.display_cart()
1019
1020     # Remove products (undo operation)
1021     print("🗑 Removing products (Undo operations):\n")
1022     cart.remove_product()
1023     cart.remove_product_front()
1024
1025     # Display updated cart
1026     cart.display_cart()
1027
1028     # Continue shopping
1029     print("🛒 Adding more products:\n")
1030     p6 = Product(106, "Headphones", 149.99, 1)
1031     cart.add_product(p6)
1032
1033     # Final cart display
1034     cart.display_cart()
1035
1036     # Summary
1037     print(f"✅ Final Total: ${cart.get_total_price():.2f}")

```

Output:

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
=====
SMART E-COMMERCE SHOPPING CART SYSTEM
=====

🛒 Adding products to cart:

✅ Added to cart: Laptop - $999.99
✅ Added to cart: Wireless Mouse - $29.99
✅ Added to cart: USB-C Cable - $15.99
⭐ Priority added to cart: Monitor - $299.99
✅ Added to cart: Keyboard - $79.99

🛒 SHOPPING CART:
-----
1. Monitor | Price: $299.99 | Qty: 1 | Subtotal: $299.99
2. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
3. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
4. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
5. Keyboard | Price: $79.99 | Qty: 1 | Subtotal: $79.99
-----
Total Price: $1487.92
-----

🗑 Removing products (Undo operations):

❌ Removed from cart: Keyboard
❌ Removed from front: Monitor

🛒 SHOPPING CART:
-----
1. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
-----
Total Price: $1107.94
-----

🛒 Adding more products:

✅ Added to cart: Headphones - $149.99

🛒 SHOPPING CART:
-----
1. Laptop | Price: $999.99 | Qty: 1 | Subtotal: $999.99
2. Wireless Mouse | Price: $29.99 | Qty: 2 | Subtotal: $59.98
3. USB-C Cable | Price: $15.99 | Qty: 3 | Subtotal: $47.97
4. Headphones | Price: $149.99 | Qty: 1 | Subtotal: $149.99
-----
Total Price: $1257.93
-----

✅ Final Total: $1257.93
PS C:\Users\arell\Music\aiac> []
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from

the list below:

- o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
 - Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

1032 """
1033 Smart Logistics & Warehouse Management
1034 Scenario:
1035 A logistics company wants to implement:
1036 1. Shipment Processing
1037 2. Urgent Shipment Priority
1038 3. Product Lookup by Code
1039 4. Warehouse Network Connectivity
1040 5. Inventory Add/Remove Operations
1041 Student Task
1042 • For each feature, select the most appropriate data structure from
1043 the list below:
1044 o Stack
1045 o Queue
1046 o Priority Queue
1047 o Linked List
1048 o Binary Search Tree (BST)
1049 o Graph
1050 o Hash Table
1051 o Deque
1052 • Justify your choice in 2-3 sentences per feature.
1053 • Implement one selected feature as a working Python program with
1054 AI-assisted code generation.
1055 Expected Output:
1056 • A table mapping feature → chosen data structure → justification.
1057 • A functional Python program implementing the chosen feature
1058 with comments and docstrings.
1059 """
1060
1061 """
1062 FEATURE MAPPING TABLE:
1063 =====
1064 1. Shipment Processing → Queue
1065     Justification: Shipments must be processed in the order they arrive for fair distribution.
1066     Queue's FIFO nature ensures shipments are handled chronologically without delays.
1067
1068 2. Urgent Shipment Priority → Priority Queue
1069     Justification: Urgent shipments need expedited handling regardless of arrival time.
1070     Priority queue allows sorting by urgency level to serve critical shipments first.
1071
1072 3. Product Lookup by Code → Hash Table
1073     Justification: Provides O(1) average lookup time using product code as key.
1074     Enables instant retrieval of product details, stock levels, and warehouse location.
1075
1076 4. Warehouse Network Connectivity → Graph
1077     Justification: Warehouses are nodes and shipping routes are edges in the network.
1078     Graph structure enables pathfinding and connectivity analysis between distribution centers.
1079
1080 5. Inventory Add/Remove Operations → Stack
1081     Justification: Uses LIFO principle for inventory management (last added items are picked first).
1082     Stack efficiently handles quick addition and removal of stock from warehouse shelves.
1083 """
1084
1085 # IMPLEMENTATION: Shipment Processing System using Queue
1086
1087 class Shipment:
1088     """Represents a shipment in the logistics system."""
1089
1090     def __init__(self, shipment_id, destination, weight, status="Pending"):
1091         self.shipment_id = shipment_id
1092         self.destination = destination
1093         self.weight = weight
1094         self.status = status
1095

```

```

1095
1096     def __repr__(self):
1097         return f"Shipment({self.shipment_id}, {self.destination}, {self.weight}kg, {self.status})"
1098
1099
1100 class ShipmentQueue:
1101     """Queue-based shipment processing system."""
1102
1103     def __init__(self):
1104         self.queue = []
1105         self.processed = []
1106
1107     def enqueue_shipment(self, shipment):
1108         """Add a shipment to the processing queue."""
1109         self.queue.append(shipment)
1110         print(f"📦 Shipment {shipment.shipment_id} queued to {shipment.destination}")
1111
1112     def process_shipment(self):
1113         """Process the next shipment in queue (FIFO)."""
1114         if not self.is_empty():
1115             shipment = self.queue.pop(0)
1116             shipment.status = "Processed"
1117             self.processed.append(shipment)
1118             print(f"🔄 Processing: {shipment.shipment_id} to {shipment.destination}")
1119             return shipment
1120         else:
1121             print("⚠️ No shipments to process.")
1122             return None
1123
1124     def peek_next_shipment(self):
1125         """View the next shipment without processing."""
1126         if not self.is_empty():
1127             shipment = self.queue[0]
1128             print(f"👁️ Next shipment: {shipment.shipment_id} to {shipment.destination}")
1129             return shipment
1130         else:
1131             print("⚠️ No shipments in queue.")
1132             return None
1133
1134     def is_empty(self):
1135         """Check if queue is empty."""
1136         return len(self.queue) == 0
1137
1138     def display_queue(self):
1139         """Display all pending shipments."""
1140         if self.is_empty():
1141             print("📭 No pending shipments.\n")
1142         else:
1143             print("\n📋 PENDING SHIPMENTS QUEUE:")
1144             print("-" * 70)
1145             for i, shipment in enumerate(self.queue, 1):
1146                 print(f"{i}. ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
1147             print("-" * 70 + "\n")
1148

```

```

1148
1149 def display_processed(self):
1150     """Display all processed shipments."""
1151     if not self.processed:
1152         print("🚚 No processed shipments yet.\n")
1153     else:
1154         print("\n📦 PROCESSED SHIPMENTS:")
1155         print("-" * 70)
1156         for i, shipment in enumerate(self.processed, 1):
1157             print(f"{i}. ID: {shipment.shipment_id} | Destination: {shipment.destination} | Weight: {shipment.weight}kg")
1158         print("-" * 70 + "\n")
1159
1160
1161 # Example Usage
1162 if __name__ == "__main__":
1163     print("=" * 70)
1164     print("🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM")
1165     print("=" * 70 + "\n")
1166
1167     system = ShipmentQueue()
1168
1169     # Create and add shipments
1170     print("📦 Adding shipments to queue:\n")
1171     s1 = Shipment("SHP-001", "New York", 50)
1172     s2 = Shipment("SHP-002", "Los Angeles", 75)
1173     s3 = Shipment("SHP-003", "Chicago", 60)
1174     s4 = Shipment("SHP-004", "Houston", 45)
1175     s5 = Shipment("SHP-005", "Miami", 55)
1176
1177     system.enqueue_shipment(s1)
1178     system.enqueue_shipment(s2)
1179     system.enqueue_shipment(s3)
1180     system.enqueue_shipment(s4)
1181     system.enqueue_shipment(s5)
1182
1183     # Display pending queue
1184     system.display_queue()
1185
1186     # Peek next shipment
1187     print("🔍 Checking next shipment:\n")
1188     system.peek_next_shipment()
1189
1190     # Process shipments
1191     print("\n🔄 PROCESSING SHIPMENTS:\n")
1192     system.process_shipment()
1193     system.process_shipment()
1194     system.process_shipment()
1195
1196     # Display updated queue
1197     system.display_queue()
1198
1199     # Display processed shipments
1200     system.display_processed()
1201
1202     # Process remaining shipments
1203     print("Processing remaining shipments:\n")
1204     system.process_shipment()
1205     system.process_shipment()
1206
1207     system.display_queue()
1208     system.display_processed()
1209

```

Output:

```
Final total: $1257.93
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"

=====
🚚 SMART LOGISTICS SHIPMENT PROCESSING SYSTEM
=====

📦 Adding shipments to queue:

✅ Shipment SHP-001 queued to New York
✅ Shipment SHP-002 queued to Los Angeles
✅ Shipment SHP-003 queued to Chicago
✅ Shipment SHP-004 queued to Houston
✅ Shipment SHP-005 queued to Miami

📋 PENDING SHIPMENTS QUEUE:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----

🔍 Checking next shipment:

📋 Next shipment: SHP-001 to New York

🔥 PROCESSING SHIPMENTS:

📦 Processing: SHP-001 to New York
📦 Processing: SHP-002 to Los Angeles
📦 Processing: SHP-003 to Chicago

📋 PENDING SHIPMENTS QUEUE:
-----
1. ID: SHP-004 | Destination: Houston | Weight: 45kg
2. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----

✅ PROCESSED SHIPMENTS:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
-----

Processing remaining shipments:

📦 Processing: SHP-004 to Houston
📦 Processing: SHP-005 to Miami
📋 No pending shipments.

✅ PROCESSED SHIPMENTS:
-----
1. ID: SHP-001 | Destination: New York | Weight: 50kg
2. ID: SHP-002 | Destination: Los Angeles | Weight: 75kg
3. ID: SHP-003 | Destination: Chicago | Weight: 60kg
4. ID: SHP-004 | Destination: Houston | Weight: 45kg
5. ID: SHP-005 | Destination: Miami | Weight: 55kg
-----

PS C:\Users\arell\Music\aiac> 
```

Task Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

1212 """
1213 FEATURE MAPPING TABLE:
1214 =====
1215 1. Customer Service Token System → Queue
1216     Justification: Customers must be served in FIFO order based on token arrival time.
1217     Queue ensures fair service and maintains chronological order without favoritism.
1218
1219 2. Loan Approval Priority Handling → Priority Queue
1220     Justification: High-value or urgent loans need expedited processing regardless of arrival time.
1221     Priority queue automatically prioritizes loans by importance, processing critical applications first.
1222
1223 3. Account Information Lookup → Hash Table
1224     Justification: Provides O(1) average lookup time using Account Number as key.
1225     Enables instant retrieval of customer details, balance, and account status.
1226
1227 4. Transaction History Management → Linked List
1228     Justification: Maintains chronological order of transactions with easy insertion/deletion.
1229     Linked list efficiently stores transaction records without requiring contiguous memory.
1230
1231 5. ATM Network Connectivity → Graph
1232     Justification: ATMs are nodes and connections between cities are edges in the network.
1233     Graph enables pathfinding and connectivity analysis for ATM availability across regions.
1234 """
1235
1236 # IMPLEMENTATION: Customer Service Token System using Queue
1237
1238 class Token:
1239     """Represents a customer service token."""
1240
1241     def __init__(self, token_id, customer_name, service_type):
1242         self.token_id = token_id
1243         self.customer_name = customer_name
1244         self.service_type = service_type
1245         self.status = "Waiting"
1246
1247     def __repr__(self):
1248         return f"Token({self.token_id}, {self.customer_name}, {self.service_type})"
1249
1250
1251 class BankServiceQueue:
1252     """Manages customer service tokens in FIFO order."""
1253
1254     def __init__(self):
1255         self.queue = []
1256         self.served_tokens = []
1257
1258     def issue_token(self, token_id, customer_name, service_type):
1259         """Issue a new service token to a customer."""
1260         token = Token(token_id, customer_name, service_type)
1261         self.queue.append(token)
1262         print(f"✅ Token #{token_id} issued to {customer_name} for {service_type}")
1263
1264     def call_next_customer(self):
1265         """Call the next customer in queue."""
1266         if not self.is_empty():
1267             token = self.queue.pop(0)
1268             token.status = "Served"
1269             self.served_tokens.append(token)
1270             print(f"📞 Calling: Token #{token.token_id} - {token.customer_name} ({token.service_type})")
1271             return token
1272         else:
1273             print("⚠️ No customers in queue.")
1274             return None
1275

```

```

1274         return None
1275
1276     def peek_next_customer(self):
1277         """Check who is the next customer without removing from queue."""
1278         if not self.is_empty():
1279             token = self.queue[0]
1280             print(f"👁️ Next customer: Token #{token.token_id} - {token.customer_name}")
1281             return token
1282         else:
1283             print("⚠️ Queue is empty.")
1284             return None
1285
1286     def is_empty(self):
1287         """Check if queue is empty."""
1288         return len(self.queue) == 0
1289
1290     def display_waiting_queue(self):
1291         """Display all customers waiting in queue."""
1292         if self.is_empty():
1293             print("🚶 No customers waiting.\n")
1294         else:
1295             print("\n👥 CUSTOMERS WAITING IN QUEUE:")
1296             print("-" * 70)
1297             for i, token in enumerate(self.queue, 1):
1298                 print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
1299             print("-" * 70 + "\n")
1300
1301     def display_served_customers(self):
1302         """Display all served customers."""
1303         if not self.served_tokens:
1304             print("👤 No customers served yet.\n")
1305         else:
1306             print("\n✅ SERVED CUSTOMERS:")
1307             print("-" * 70)
1308             for i, token in enumerate(self.served_tokens, 1):
1309                 print(f"{i}. Token #{token.token_id} | {token.customer_name} | {token.service_type}")
1310             print("-" * 70 + "\n")
1311
1312
1313 # Example Usage
1314 if __name__ == "__main__":
1315     print("-" * 70)
1316     print("🏦 SMART BANKING CUSTOMER SERVICE SYSTEM")
1317     print("-" * 70 + "\n")
1318
1319     bank = BankServiceQueue()
1320
1321     # Issue tokens to customers
1322     print("📄 Issuing service tokens:\n")
1323     bank.issue_token(101, "Alice Johnson", "Deposit")
1324     bank.issue_token(102, "Bob Smith", "Withdrawal")
1325     bank.issue_token(103, "Carol White", "Loan Application")
1326     bank.issue_token(104, "David Brown", "Account Opening")
1327     bank.issue_token(105, "Eve Davis", "Balance Inquiry")
1328
1329     # Display waiting queue
1330     bank.display_waiting_queue()
1331
1332     # Check next customer
1333     print("👁️ Checking next customer:\n")
1334     bank.peek_next_customer()
1335
1336     # Serve customers
1337     print("\n👤 SERVING CUSTOMERS:\n")
1338     bank.call_next_customer()
1339     bank.call_next_customer()
1340     bank.call_next_customer()
1341
1342     # Display updated queue
1343     bank.display_waiting_queue()
1344
1345     # Display served customers
1346     bank.display_served_customers()
1347
1348     # Serve remaining customers
1349     print("Serving remaining customers:\n")
1350     bank.call_next_customer()
1351     bank.call_next_customer()
1352
1353     bank.display_waiting_queue()
1354     bank.display_served_customers()
1355

```

```
🟡 Issuing service tokens:

✅ Token #101 issued to Alice Johnson for Deposit
✅ Token #102 issued to Bob Smith for Withdrawal
✅ Token #103 issued to Carol White for Loan Application
✅ Token #104 issued to David Brown for Account Opening
✅ Token #105 issued to Eve Davis for Balance Inquiry
```

```
👤 CUSTOMERS WAITING IN QUEUE:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
🔍 Checking next customer:
```

```
📄 Next customer: Token #101 - Alice Johnson
```

```
📞 SERVING CUSTOMERS:
```

```
📞 Calling: Token #101 - Alice Johnson (Deposit)
📞 Calling: Token #102 - Bob Smith (Withdrawal)
📞 Calling: Token #103 - Carol White (Loan Application)
```

```
👤 CUSTOMERS WAITING IN QUEUE:
```

```
-----
1. Token #104 | David Brown | Account Opening
2. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
✅ SERVED CUSTOMERS:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
-----
```

```
Serving remaining customers:
```

```
📞 Calling: Token #104 - David Brown (Account Opening)
📞 Calling: Token #105 - Eve Davis (Balance Inquiry)
🟢 No customers waiting.
```

```
✅ SERVED CUSTOMERS:
```

```
-----
1. Token #101 | Alice Johnson | Deposit
2. Token #102 | Bob Smith | Withdrawal
3. Token #103 | Carol White | Loan Application
4. Token #104 | David Brown | Account Opening
5. Token #105 | Eve Davis | Balance Inquiry
-----
```

```
PS C:\Users\arell\Music\aiac> 
```

Task Description #15: Online Learning Management System – Data

Structure Selection

An online education platform wants to develop a Learning Management

System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

Code:

```

1362 FEATURE MAPPING TABLE:
1363 =====
1364 1. Student Login Authentication → Hash Table
1365 | Justification: Provides O(1) average lookup time using username/email as key.
1366 | Enables instant credential validation and fast user authentication without delays.
1367
1368 2. Assignment Submission Queue → Queue
1369 | Justification: Submissions must be processed in FIFO order based on upload time.
1370 | Queue ensures fair evaluation and maintains chronological order of submissions.
1371
1372 3. Top Performer Ranking → Priority Queue
1373 | Justification: Students ranked by scores with highest performers prioritized.
1374 | Priority queue automatically maintains sorted order for quick access to top performers.
1375
1376 4. Course Prerequisite Structure → Graph
1377 | Justification: Courses are nodes and prerequisites are directed edges between them.
1378 | Graph enables validation of course dependencies and pathfinding for course sequences.
1379
1380 5. Undo Feature in Online Editor → Stack
1381 | Justification: Uses LIFO principle to reverse actions in reverse order of execution.
1382 | Stack efficiently stores action history and allows quick undo operations.
1383 """
1384
1385 # IMPLEMENTATION: Assignment Submission Queue using Queue
1386
1387 class Assignment:
1388
1389     def __init__(self, submission_id, student_name, assignment_name, submission_time, score=None):
1390         self.submission_id = submission_id
1391         self.student_name = student_name
1392         self.assignment_name = assignment_name
1393         self.submission_time = submission_time
1394         self.score = score
1395         self.status = "Pending"
1396
1397     def __repr__(self):
1398         return f"Assignment({self.submission_id}, {self.student_name}, {self.assignment_name})"
1399
1400
1401 class AssignmentSubmissionQueue:
1402
1403     def __init__(self):
1404         self.queue = []
1405         self.evaluated = []
1406

```

```

1407 def submit_assignment(self, submission_id, student_name, assignment_name, submission_time):
1408     assignment = Assignment(submission_id, student_name, assignment_name, submission_time)
1409     self.queue.append(assignment)
1410     print(f"📄 Assignment submitted: {student_name} - {assignment_name} at {submission_time}")
1411
1412 def evaluate_next_assignment(self, score):
1413     if not self.is_empty():
1414         assignment = self.queue.pop(0)
1415         assignment.status = "Evaluated"
1416         assignment.score = score
1417         self.evaluated.append(assignment)
1418         print(f"📄 Evaluated: {assignment.student_name}'s {assignment.assignment_name} (Score: {score}/100)")
1419         return assignment
1420     else:
1421         print("⚠️ No assignments to evaluate.")
1422         return None
1423
1424 def peek_next_assignment(self):
1425     if not self.is_empty():
1426         assignment = self.queue[0]
1427         print(f"📄 Next assignment: {assignment.student_name} - {assignment.assignment_name}")
1428         return assignment
1429     else:
1430         print("⚠️ Queue is empty.")
1431         return None
1432
1433 def is_empty(self):
1434     return len(self.queue) == 0
1435
1436 def display_pending_queue(self):
1437     if self.is_empty():
1438         print("📄 No pending assignments.\n")
1439     else:
1440         print("\n📄 PENDING ASSIGNMENTS QUEUE:")
1441         print("-" * 80)
1442         for i, assignment in enumerate(self.queue, 1):
1443             print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | {assignment.submission_time}")
1444         print("-" * 80 + "\n")
1445
1446 def display_evaluated_assignments(self):
1447     if not self.evaluated:
1448         print("📄 No evaluated assignments yet.\n")
1449     else:
1450         print("\n📄 EVALUATED ASSIGNMENTS:")
1451         print("-" * 80)
1452         for i, assignment in enumerate(self.evaluated, 1):
1453             print(f"{i}. ID: {assignment.submission_id} | {assignment.student_name} | {assignment.assignment_name} | Score: {assignment.score}/100")
1454         print("-" * 80 + "\n")

```

```

1456
1457 # Example Usage
1458 if __name__ == "__main__":
1459     print("=" * 80)
1460     print("🚀 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE")
1461     print("=" * 80 + "\n")
1462
1463     system = AssignmentSubmissionQueue()
1464
1465     # Submit assignments
1466     print("📁 Submitting assignments:\n")
1467     system.submit_assignment(1001, "Alice Johnson", "Python Basics", "09:15 AM")
1468     system.submit_assignment(1002, "Bob Smith", "Data Structures", "09:30 AM")
1469     system.submit_assignment(1003, "Carol White", "Python Basics", "09:45 AM")
1470     system.submit_assignment(1004, "David Brown", "Web Development", "10:00 AM")
1471     system.submit_assignment(1005, "Eve Davis", "Data Structures", "10:15 AM")
1472
1473     # Display pending queue
1474     system.display_pending_queue()
1475
1476     # Check next assignment
1477     print("🔍 Checking next assignment to evaluate:\n")
1478     system.peek_next_assignment()
1479
1480     # Evaluate assignments
1481     print("\n📋 EVALUATING ASSIGNMENTS:\n")
1482     system.evaluate_next_assignment(92)
1483     system.evaluate_next_assignment(85)
1484     system.evaluate_next_assignment(88)
1485
1486     # Display updated queue
1487     system.display_pending_queue()
1488
1489     # Display evaluated assignments
1490     system.display_evaluated_assignments()
1491
1492     # Evaluate remaining assignments
1493     print("Evaluating remaining assignments:\n")
1494     system.evaluate_next_assignment(95)
1495     system.evaluate_next_assignment(78)
1496
1497     system.display_pending_queue()
1498     system.display_evaluated_assignments()

```

Output:

=====

🔍 ONLINE LEARNING MANAGEMENT SYSTEM - ASSIGNMENT SUBMISSION QUEUE

=====

📁 Submitting assignments:

- ✅ Assignment submitted: Alice Johnson - Python Basics at 09:15 AM
- ✅ Assignment submitted: Bob Smith - Data Structures at 09:30 AM
- ✅ Assignment submitted: Carol White - Python Basics at 09:45 AM
- ✅ Assignment submitted: David Brown - Web Development at 10:00 AM
- ✅ Assignment submitted: Eve Davis - Data Structures at 10:15 AM

📁 PENDING ASSIGNMENTS QUEUE:

-
1. ID: 1001 | Alice Johnson | Python Basics | 09:15 AM
 2. ID: 1002 | Bob Smith | Data Structures | 09:30 AM
 3. ID: 1003 | Carol White | Python Basics | 09:45 AM
 4. ID: 1004 | David Brown | Web Development | 10:00 AM
 5. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
-

🔍 Checking next assignment to evaluate:

📁 Next assignment: Alice Johnson - Python Basics

📁 EVALUATING ASSIGNMENTS:

- 📁 Evaluated: Alice Johnson's Python Basics (Score: 92/100)
- 📁 Evaluated: Bob Smith's Data Structures (Score: 85/100)
- 📁 Evaluated: Carol White's Python Basics (Score: 88/100)

📁 PENDING ASSIGNMENTS QUEUE:

-
1. ID: 1004 | David Brown | Web Development | 10:00 AM
 2. ID: 1005 | Eve Davis | Data Structures | 10:15 AM
-

✅ EVALUATED ASSIGNMENTS:

-
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
 2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
 3. ID: 1003 | Carol White | Python Basics | Score: 88/100
-

Evaluating remaining assignments:

- 📁 Evaluated: David Brown's Web Development (Score: 95/100)
- 📁 Evaluated: Eve Davis's Data Structures (Score: 78/100)
- 📁 No pending assignments.

✅ EVALUATED ASSIGNMENTS:

-
1. ID: 1001 | Alice Johnson | Python Basics | Score: 92/100
 2. ID: 1002 | Bob Smith | Data Structures | Score: 85/100
 3. ID: 1003 | Carol White | Python Basics | Score: 88/100
 4. ID: 1004 | David Brown | Web Development | Score: 95/100
 5. ID: 1005 | Eve Davis | Data Structures | Score: 78/100
-

PS C:\Users\arell\Music\aiac>

Task Description #16: Smart Airport Management System – Data

Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

Code:

```

1499
1500 """
1501 Smart Airport Management System | Data Structure Challenge
1502
1503 An airport authority plans to implement:
1504 1. Passenger Check-In Queue | Process passengers in order of arrival.
1505 2. Emergency Landing Priority | Aircraft in emergency get priority.
1506 3. Flight Information Lookup | Fast search using Flight ID.
1507 4. Air Route Connectivity | Airports connected via routes.
1508 5. Boarding Process (Last-In First-Out Cabin Storage) | Overhead luggage management simulation.
1509
1510 Student Task:
1511 • Choose suitable data structures from the provided list.
1512 • Provide 2-3 sentence justification per feature.
1513 • Implement one feature in Python with comments and docstrings using AI assistance.
1514
1515 Expected Output:
1516 • Mapping table.
1517 • Working Python implementation.
1518 """
1519
1520 # FEATURE MAPPING TABLE
1521 FEATURE_MAPPING = [
1522     (
1523         "Passenger Check-In Queue",
1524         "Queue",
1525         "Passengers must be processed in FIFO order based on arrival time. "
1526         "Queue ensures fair service and maintains chronological processing without delays."
1527     ),
1528     (
1529         "Emergency Landing Priority",
1530         "Priority Queue",
1531         "Aircraft in emergency situations require immediate landing priority. "
1532         "Priority queue sorts by urgency level, ensuring critical aircraft land first."
1533     ),
1534     (
1535         "Flight Information Lookup",
1536         "Hash Table",
1537         "Provides O(1) average lookup time using Flight ID as key. "
1538         "Enables instant retrieval of flight details like schedule, gate, and status."
1539     ),
1540     (
1541         "Air Route Connectivity",
1542         "Graph",
1543         "Airports are nodes and flight routes are edges connecting them. "
1544         "Graph structure enables pathfinding and connectivity analysis for route planning."
1545     ),
1546     (
1547         "Boarding Process (Cabin Storage)",

```

```

1555 def print_feature_mapping_table():
1556     """Display the feature mapping table with justifications."""
1557     print("\n" + "=" * 130)
1558     print("SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE")
1559     print("=" * 130)
1560     print(f"{'Feature':<35} {'Data Structure':<18} {'Justification':<75}")
1561     print("-" * 130)
1562     for feature, structure, justification in FEATURE_MAPPING:
1563         print(f"{'feature':<35} {'structure':<18} {'justification':<75}")
1564     print("=" * 130 + "\n")
1565
1566
1567 # IMPLEMENTATION: Passenger Check-In Queue using Queue Data Structure
1568
1569 class Passenger:
1570     """Represents a passenger with check-in details."""
1571
1572     def __init__(self, passenger_id, name, flight_id, arrival_time):
1573         """
1574         Initialize a passenger object.
1575
1576         Args:
1577             passenger_id: Unique identifier for the passenger
1578             name: Name of the passenger
1579             flight_id: Flight ID for the passenger's booking
1580             arrival_time: Time of arrival at check-in counter
1581         """
1582         self.passenger_id = passenger_id
1583         self.name = name
1584         self.flight_id = flight_id
1585         self.arrival_time = arrival_time
1586         self.status = "Waiting"
1587
1588     def __repr__(self):
1589         return f"Passenger({self.passenger_id}, {self.name}, Flight: {self.flight_id}, Time: {self.arrival_time})"
1590
1591
1592 class PassengerCheckInQueue:
1593     """Manages passenger check-in queue using FIFO principle."""
1594
1595     def __init__(self):
1596         """Initialize the check-in queue and tracking lists."""
1597         self.queue = []
1598         self.checked_in = []
1599
1600     def add_passenger(self, passenger_id, name, flight_id, arrival_time):
1601         """
1602         Add a passenger to the check-in queue.
1603
1604         Args:
1605             passenger_id: Unique passenger identifier

```

```

1605         passenger_id: Unique passenger identifier
1606         name: Passenger name
1607         flight_id: Flight ID
1608         arrival_time: Arrival time at counter
1609     """
1610     passenger = Passenger(passenger_id, name, flight_id, arrival_time)
1611     self.queue.append(passenger)
1612     print(f"✅ Passenger added to queue: {name} (ID: {passenger_id}) - Flight {flight_id}")
1613
1614     def process_check_in(self):
1615         """
1616         Process the next passenger in queue (FIFO).
1617
1618         Returns:
1619             Passenger object if processed, None if queue is empty
1620         """
1621         if not self.is_empty():
1622             passenger = self.queue.pop(0)
1623             passenger.status = "Checked-In"
1624             self.checked_in.append(passenger)
1625             print(f"✅ Checked-In: {passenger.name} (Flight: {passenger.flight_id})")
1626             return passenger
1627         else:
1628             print("⚠️ No passengers in queue.")
1629             return None
1630
1631     def peek_next_passenger(self):
1632         """
1633         View the next passenger without removing from queue.
1634
1635         Returns:
1636             Next passenger object if available, None if queue is empty
1637         """
1638         if not self.is_empty():
1639             passenger = self.queue[0]
1640             print(f"👁️ Next passenger: {passenger.name} (Flight: {passenger.flight_id})")
1641             return passenger
1642         else:
1643             print("⚠️ Queue is empty.")
1644             return None
1645
1646     def is_empty(self):
1647         """Check if the queue is empty."""
1648         return len(self.queue) == 0
1649

```

```

1592 class PassengerCheckInQueue:
1654     def display_waiting_passengers(self):
1655         """Display all passengers currently waiting in the queue."""
1656         if self.is_empty():
1657             print("✋ No passengers waiting.\n")
1658         else:
1659             print("\n📋 PASSENGER CHECK-IN QUEUE (FIFO):")
1660             print("-" * 100)
1661             for i, passenger in enumerate(self.queue, 1):
1662                 print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Arrival: {passenger.arrival_time}")
1663             print("-" * 100 + "\n")
1664
1665     def display_checked_in_passengers(self):
1666         """Display all passengers who have been checked in."""
1667         if not self.checked_in:
1668             print("✋ No passengers checked-in yet.\n")
1669         else:
1670             print("\n✅ CHECKED-IN PASSENGERS:")
1671             print("-" * 100)
1672             for i, passenger in enumerate(self.checked_in, 1):
1673                 print(f"{i}. ID: {passenger.passenger_id:<6} | Name: {passenger.name:<20} | Flight: {passenger.flight_id:<8} | Status: {passenger.status}")
1674             print("-" * 100 + "\n")
1675
1676
1677 # Example Usage
1678 if __name__ == "__main__":
1679     # Display feature mapping table
1680     print_feature_mapping_table()
1681
1682     print("-" * 100)
1683     print("✈️ SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE")
1684     print("-" * 100 + "\n")
1685
1686     # Create check-in system
1687     system = PassengerCheckInQueue()
1688
1689     # Add passengers to queue
1690     print("📋 Adding passengers to check-in queue:\n")
1691     system.add_passenger(101, "Alice Johnson", "AA-101", "08:00 AM")
1692     system.add_passenger(102, "Bob Smith", "UA-205", "08:05 AM")
1693     system.add_passenger(103, "Carol White", "DL-310", "08:10 AM")
1694     system.add_passenger(104, "David Brown", "AA-101", "08:15 AM")
1695     system.add_passenger(105, "Eve Davis", "UA-205", "08:20 AM")
1696     system.add_passenger(106, "Frank Miller", "SW-415", "08:25 AM")
1697
1698     # Display waiting passengers
1699     system.display_waiting_passengers()
1700
1701     # Peek at next passenger
1702     print("👁️ Checking next passenger:\n")

```

```

assg_11.py > ...
1697
1698     # Display waiting passengers
1699     system.display_waiting_passengers()
1700
1701     # Peek at next passenger
1702     print("👁️ Checking next passenger:\n")
1703     system.peak_next_passenger()
1704     print(f"\n📋 Queue Length: {system.get_queue_length()} passengers waiting\n")
1705
1706     # Process check-ins
1707     print("✈️ PROCESSING CHECK-INS:\n")
1708     system.process_check_in()
1709     system.process_check_in()
1710     system.process_check_in()
1711
1712     # Display updated queue
1713     system.display_waiting_passengers()
1714     print(f"📋 Remaining Queue Length: {system.get_queue_length()} passengers\n")
1715
1716     # Display checked-in passengers
1717     system.display_checked_in_passengers()
1718
1719     # Continue processing
1720     print("Continuing to process remaining passengers:\n")
1721     system.process_check_in()
1722     system.process_check_in()
1723     system.process_check_in()
1724
1725     # Final displays
1726     system.display_waiting_passengers()
1727     system.display_checked_in_passengers()
1728
1729     print("✈️ All passengers have been processed!")

```

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
```

SMART AIRPORT MANAGEMENT SYSTEM - FEATURE MAPPING TABLE

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	Passengers must be processed in FIFO order based on arrival time. Queue ensures fair service and maintains chronological processing without delays.
Emergency Landing Priority	Priority Queue	Aircraft in emergency situations require immediate landing priority. Priority queue sorts by urgency level, ensuring critical aircraft land first.
Flight Information Lookup	Hash Table	Provides O(1) average lookup time using Flight ID as key. Enables instant retrieval of flight details like schedule, gate, and status.
Air Route Connectivity	Graph	Airports are nodes and flight routes are edges connecting them. Graph structure enables pathfinding and connectivity analysis for route planning.
Boarding Process (Cabin Storage)	Stack	Uses LIFO principle for overhead luggage management. Stack efficiently handles last-loaded items being retrieved first during deplaning.

SMART AIRPORT MANAGEMENT SYSTEM - PASSENGER CHECK-IN QUEUE

Adding passengers to check-in queue:

- ✓ Passenger added to queue: Alice Johnson (ID: 101) - Flight AA-101
- ✓ Passenger added to queue: Bob Smith (ID: 102) - Flight UA-205
- ✓ Passenger added to queue: Carol White (ID: 103) - Flight DL-310
- ✓ Passenger added to queue: David Brown (ID: 104) - Flight AA-101
- ✓ Passenger added to queue: Eve Davis (ID: 105) - Flight UA-205
- ✓ Passenger added to queue: Frank Miller (ID: 106) - Flight SW-415

PASSENGER CHECK-IN QUEUE (FIFO):

1. ID: 101	Name: Alice Johnson	Flight: AA-101	Arrival: 08:00 AM
2. ID: 102	Name: Bob Smith	Flight: UA-205	Arrival: 08:05 AM
3. ID: 103	Name: Carol White	Flight: DL-310	Arrival: 08:10 AM
4. ID: 104	Name: David Brown	Flight: AA-101	Arrival: 08:15 AM
5. ID: 105	Name: Eve Davis	Flight: UA-205	Arrival: 08:20 AM
6. ID: 106	Name: Frank Miller	Flight: SW-415	Arrival: 08:25 AM

Checking next passenger:

Next passenger: Alice Johnson (Flight: AA-101)

Queue Length: 6 passengers waiting

PROCESSING CHECK-INS:

- ✓ Checked-In: Alice Johnson (Flight: AA-101)
- ✓ Checked-In: Bob Smith (Flight: UA-205)
- ✓ Checked-In: Carol White (Flight: DL-310)

PASSENGER CHECK-IN QUEUE (FIFO):

1. ID: 104	Name: David Brown	Flight: AA-101	Arrival: 08:15 AM
2. ID: 105	Name: Eve Davis	Flight: UA-205	Arrival: 08:20 AM
3. ID: 106	Name: Frank Miller	Flight: SW-415	Arrival: 08:25 AM

Remaining Queue Length: 3 passengers

✓ CHECKED-IN PASSENGERS:

1. ID: 101	Name: Alice Johnson	Flight: AA-101	Status: Checked-In
2. ID: 102	Name: Bob Smith	Flight: UA-205	Status: Checked-In
3. ID: 103	Name: Carol White	Flight: DL-310	Status: Checked-In

Continuing to process remaining passengers:

- ✓ Checked-In: David Brown (Flight: AA-101)
- ✓ Checked-In: Eve Davis (Flight: UA-205)
- ✓ Checked-In: Frank Miller (Flight: SW-415)
- ✈ No passengers waiting.

✓ CHECKED-IN PASSENGERS:

1. ID: 101	Name: Alice Johnson	Flight: AA-101	Status: Checked-In
2. ID: 102	Name: Bob Smith	Flight: UA-205	Status: Checked-In
3. ID: 103	Name: Carol White	Flight: DL-310	Status: Checked-In
4. ID: 104	Name: David Brown	Flight: AA-101	Status: Checked-In
5. ID: 105	Name: Eve Davis	Flight: UA-205	Status: Checked-In
6. ID: 106	Name: Frank Miller	Flight: SW-415	Status: Checked-In

✈ All passengers have been processed!

PS C:\Users\arell\Music\aiac>

Task Description #17: Smart Social Media Platform – Data Structure

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

Code:

```

1713 FEATURE MAPPING TABLE:
1714 =====
1715 1. News Feed Generation → Linked List
1716 | Justification: Maintains chronological order of posts with easy insertion at the beginning.
1717 | Linked list efficiently handles dynamic post addition without requiring contiguous memory.
1718
1719 2. Trending Hashtags Tracker → Priority Queue
1720 | Justification: Hashtags ranked by usage frequency with most popular appearing first.
1721 | Priority queue automatically maintains sorted order for quick access to trending topics.
1722
1723 3. User Profile Search → Hash Table
1724 | Justification: Provides O(1) average lookup time using username as key.
1725 | Enables instant retrieval of user profiles and account information.
1726
1727 4. Friend Network Representation → Graph
1728 | Justification: Users are nodes and friend connections are edges in the network.
1729 | Graph structure enables pathfinding and connectivity analysis for friend recommendations.
1730
1731 5. Message Undo Feature → Stack
1732 | Justification: Uses LIFO principle to reverse messages in reverse order of sending.
1733 | Stack efficiently stores message history and allows quick undo operations.
1734 """
1735
1736 # IMPLEMENTATION: Trending Hashtags Tracker using Priority Queue
1737
1738
1739
1740 class Hashtag:
1741     """Represents a hashtag with its usage count."""
1742
1743     def __init__(self, tag_name, usage_count=1):
1744         self.tag_name = tag_name
1745         self.usage_count = usage_count
1746
1747     def __lt__(self, other):
1748         """Compare hashtags by usage count (higher count = higher priority)."""
1749         return self.usage_count > other.usage_count
1750
1751     def __repr__(self):
1752         return f"#{self.tag_name} ({self.usage_count})"
1753
1754
1755 class TrendingHashtagTracker:
1756     """
1757     Manages trending hashtags using a Priority Queue (min-heap).
1758     Higher usage count = higher priority = more trending.
1759     """
1760
1761     def __init__(self):
1762         self.heap = []
1763         self.hashtag_dict = {}
1764

```

```

1765 def add_hashtag(self, tag_name):
1766     """
1767     Add a hashtag or increment its usage count.
1768
1769     Args:
1770     | tag_name: Name of the hashtag (without #)
1771     """
1772     if tag_name in self.hashtag_dict:
1773         self.hashtag_dict[tag_name] += 1
1774         print(f"🔄 Updated #{tag_name} - Usage count: {self.hashtag_dict[tag_name]}")
1775     else:
1776         self.hashtag_dict[tag_name] = 1
1777         print(f"✅ Added new hashtag #{tag_name}")
1778
1779 def get_trending_hashtags(self, limit=5):
1780     """
1781     Retrieve top trending hashtags.
1782
1783     Args:
1784     | limit: Number of top hashtags to return
1785
1786     Returns:
1787     | List of top trending hashtags
1788     """
1789     heap = []
1790     for tag_name, count in self.hashtag_dict.items():
1791         heapq.heappush(heap, (-count, tag_name))
1792
1793     trending = []
1794     for _ in range(min(limit, len(heap))):
1795         count, tag_name = heapq.heappop(heap)
1796         trending.append((tag_name, -count))
1797
1798     return trending
1799
1800 def display_top_trending(self, limit=5):
1801     """Display top trending hashtags."""
1802     trending = self.get_trending_hashtags(limit)
1803
1804     if not trending:
1805         print("🚫 No hashtags yet.\n")
1806     else:
1807         print(f"\n🔥 TOP {limit} TRENDING HASHTAGS:")
1808         print("-" * 60)
1809         for i, (tag_name, count) in enumerate(trending, 1):
1810             print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times | 📊 {'█' * (count // 5)}")
1811         print("-" * 60 + "\n")

```

```

1812
1813 def display_all_hashtags(self):
1814     """Display all hashtags sorted by usage count."""
1815     if not self.hashtag_dict:
1816         print("🚫 No hashtags yet.\n")
1817     else:
1818         sorted_tags = sorted(self.hashtag_dict.items(), key=lambda x: x[1], reverse=True)
1819         print("\n📋 ALL HASHTAGS (Sorted by Usage):")
1820         print("-" * 60)
1821         for i, (tag_name, count) in enumerate(sorted_tags, 1):
1822             print(f"{i}. #{tag_name:<20} | Usage: {count:>3} times")
1823         print("-" * 60 + "\n")
1824
1825 def remove_hashtag(self, tag_name):
1826     """Remove a hashtag."""
1827     if tag_name in self.hashtag_dict:
1828         del self.hashtag_dict[tag_name]
1829         print(f"❌ Removed hashtag #{tag_name}")
1830     else:
1831         print(f"⚠️ Hashtag #{tag_name} not found.")
1832
1833

```

```

1833
1834 # Example Usage
1835 if __name__ == "__main__":
1836     print("=" * 60)
1837     print("📊 SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER")
1838     print("=" * 60 + "\n")
1839
1840     tracker = TrendingHashtagTracker()
1841
1842     # Add hashtags with multiple uses
1843     print("📄 Adding hashtags from posts:\n")
1844     tracker.add_hashtag("Python")
1845     tracker.add_hashtag("DataStructures")
1846     tracker.add_hashtag("Python")
1847     tracker.add_hashtag("Coding")
1848     tracker.add_hashtag("Python")
1849     tracker.add_hashtag("WebDevelopment")
1850     tracker.add_hashtag("DataStructures")
1851     tracker.add_hashtag("Python")
1852     tracker.add_hashtag("AI")
1853     tracker.add_hashtag("Coding")
1854     tracker.add_hashtag("Python")
1855     tracker.add_hashtag("AI")
1856
1857     # Display all hashtags
1858     tracker.display_all_hashtags()
1859
1860     # Display top trending
1861     print("🔥 Checking trending hashtags:\n")
1862     tracker.display_top_trending(5)
1863
1864     # Add more activity to a hashtag
1865     print("📄 More posts using hashtags:\n")
1866     tracker.add_hashtag("AI")
1867     tracker.add_hashtag("AI")
1868     tracker.add_hashtag("AI")
1869
1870     # Display updated trending
1871     tracker.display_top_trending(5)
1872
1873     # Remove a hashtag
1874     print("🗑 Removing hashtag:\n")
1875     tracker.remove_hashtag("WebDevelopment")
1876
1877     # Final display
1878     tracker.display_all_hashtags()

```

```

PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
=====
SMART SOCIAL MEDIA PLATFORM - TRENDING HASHTAGS TRACKER
=====

📁 Adding hashtags from posts:

✅ Added new hashtag #Python
✅ Added new hashtag #DataStructures
❌ Updated #Python - Usage count: 2
✅ Added new hashtag #Coding
❌ Updated #Python - Usage count: 3
✅ Added new hashtag #WebDevelopment
❌ Updated #DataStructures - Usage count: 2
❌ Updated #Python - Usage count: 4
✅ Added new hashtag #AI
❌ Updated #Coding - Usage count: 2
❌ Updated #Python - Usage count: 5
❌ Updated #AI - Usage count: 2

📁 ALL HASHTAGS (Sorted by Usage):
-----
1. #Python           | Usage: 5 times
2. #DataStructures   | Usage: 2 times
3. #Coding           | Usage: 2 times
4. #AI              | Usage: 2 times
5. #WebDevelopment   | Usage: 1 times
-----

🔥 Checking trending hashtags:

🔥 TOP 5 TRENDING HASHTAGS:
-----
1. #Python           | Usage: 5 times | 📈
2. #AI              | Usage: 2 times | 📈
3. #Coding           | Usage: 2 times | 📈
4. #DataStructures   | Usage: 2 times | 📈
5. #WebDevelopment   | Usage: 1 times | 📈
-----

📁 More posts using hashtags:

❌ Updated #AI - Usage count: 3
❌ Updated #AI - Usage count: 4
❌ Updated #AI - Usage count: 5

🔥 TOP 5 TRENDING HASHTAGS:
-----
1. #AI              | Usage: 5 times | 📈
2. #Python          | Usage: 5 times | 📈
3. #Coding           | Usage: 2 times | 📈
4. #DataStructures   | Usage: 2 times | 📈
5. #WebDevelopment   | Usage: 1 times | 📈
-----

🗑 Removing hashtag:

❌ Removed hashtag #WebDevelopment

📁 ALL HASHTAGS (Sorted by Usage):
-----
1. #Python           | Usage: 5 times
2. #AI              | Usage: 5 times
3. #DataStructures   | Usage: 2 times
4. #Coding           | Usage: 2 times
-----

PS C:\Users\arell\Music\aiac>

```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.

- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

1871 """
1872 FEATURE MAPPING TABLE:
1873 =====
1874 1. Room Allotment → Queue
1875     Justification: Students are assigned rooms in the order they apply.
1876     Queue ensures fair allocation and maintains chronological order without bias.
1877
1878 2. Medical Emergency Handling → Priority Queue (implemented without heapq)
1879     Justification: Critical patients need expedited attention regardless of arrival time.
1880     Priority queue sorts by severity level to serve emergencies in order of urgency.
1881
1882 3. Student Record Retrieval → Hash Table
1883     Justification: Provides O(1) average lookup time using roll number as key.
1884     Enables instant retrieval of student details and academic records.
1885
1886 4. Hostel Block Connectivity → Graph
1887     Justification: Blocks are nodes and connections between blocks are edges.
1888     Graph structure enables pathfinding and connectivity analysis for navigation.
1889
1890 5. Visitor Entry Log → Linked List
1891     Justification: Maintains chronological order of visitor entries with easy insertion.
1892     Linked list efficiently handles dynamic logging without requiring contiguous memory.
1893 """
1894
1895 # IMPLEMENTATION: Medical Emergency Handling using Priority Queue (without heapq)
1896
1897 class Patient:
1898
1899     def __init__(self, patient_id, name, severity, arrival_time):
1900         self.patient_id = patient_id
1901         self.name = name
1902         self.severity = severity # 1=Critical, 2=High, 3=Medium, 4=Low
1903         self.arrival_time = arrival_time
1904         self.status = "Waiting"
1905
1906     def __repr__(self):
1907         return f"Patient({self.patient_id}, {self.name}, Severity: {self.severity}, Time: {self.arrival_time})"
1908
1909
1910 class MedicalEmergencyQueue:
1911
1912     def __init__(self):
1913         self.queue = []
1914         self.treated = []
1915
1916     def add_patient(self, patient_id, name, severity, arrival_time):
1917         patient = Patient(patient_id, name, severity, arrival_time)
1918         self.queue.append(patient)
1919         # Sort queue by severity (lower number = higher priority)
1920         self.queue.sort(key=lambda p: p.severity)
1921         severity_map = {1: "🔴 CRITICAL", 2: "🟠 HIGH", 3: "🟡 MEDIUM", 4: "🟢 LOW"}
1922         print(f"✅ Patient Added: {name} (ID: {patient_id}) - {severity_map[severity]}")
1923
1924     def treat_next_patient(self):
1925         if not self.is_empty():
1926             patient = self.queue.pop(0)
1927             patient.status = "Treated"
1928             self.treated.append(patient)
1929             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1930             print(f"🏥 Now Treating: {patient.name} (Severity: {severity_map[patient.severity]})")
1931             return patient
1932         else:

```

```

1910 class MedicalEmergencyQueue:
1934     return None
1935
1936     def peek_next_patient(self):
1937         if not self.is_empty():
1938             patient = self.queue[0]
1939             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1940             print(f"🏥 Next Patient: {patient.name} (Severity: {severity_map[patient.severity]}))")
1941             return patient
1942         else:
1943             print("❌ No patients in queue.")
1944             return None
1945
1946     def is_empty(self):
1947         return len(self.queue) == 0
1948
1949     def display_queue(self):
1950         if self.is_empty():
1951             print("🏥 No patients waiting.\n")
1952         else:
1953             print("\n🏥 EMERGENCY QUEUE (Sorted by Priority):")
1954             print("-" * 80)
1955             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1956             for patient in self.queue:
1957                 print(f"ID: {patient.patient_id}<6) | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]:<10} | Time: {patient.arrival_time}")
1958             print("-" * 80 + "\n")
1959
1960     def display_treated_patients(self):
1961         if not self.treated:
1962             print("🏥 No treated patients yet.\n")
1963         else:
1964             print("\n🏥 TREATED PATIENTS:")
1965             print("-" * 80)
1966             severity_map = {1: "CRITICAL", 2: "HIGH", 3: "MEDIUM", 4: "LOW"}
1967             for patient in self.treated:
1968                 print(f"ID: {patient.patient_id}<6) | Name: {patient.name:<20} | Severity: {severity_map[patient.severity]}")
1969             print("-" * 80 + "\n")

```

```

1972 # Example Usage
1973 if __name__ == "__main__":
1974     print("-" * 80)
1975     print("🏥 SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM")
1976     print("-" * 80 + "\n")
1977
1978     hospital = MedicalEmergencyQueue()
1979
1980     # Add patients with different severity levels
1981     print("🏥 Adding Patients to Emergency Queue:\n")
1982     hospital.add_patient(1, "John Doe", 4, "09:00")
1983     hospital.add_patient(2, "Sarah Smith", 1, "09:05")
1984     hospital.add_patient(3, "Mike Johnson", 3, "09:10")
1985     hospital.add_patient(4, "Emily Brown", 2, "09:15")
1986     hospital.add_patient(5, "David Wilson", 1, "09:20")
1987
1988     # Display queue
1989     hospital.display_queue()
1990
1991     # Peek next patient
1992     print("🏥 Checking Next Patient:\n")
1993     hospital.peek_next_patient()
1994
1995     # Treat patients
1996     print("\n🏥 TREATMENT PROCESS:\n")
1997     hospital.treat_next_patient()
1998     hospital.treat_next_patient()
1999     hospital.treat_next_patient()
2000
2001     # Display remaining queue
2002     hospital.display_queue()
2003
2004     # Display treated patients
2005     hospital.display_treated_patients()
2006
2007     # Treat remaining patients
2008     print("Treating remaining patients:\n")
2009     hospital.treat_next_patient()
2010     hospital.treat_next_patient()
2011
2012     hospital.display_treated_patients()

```

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
=====
SMART UNIVERSITY HOSTEL MEDICAL EMERGENCY SYSTEM
=====

■ Adding Patients to Emergency Queue:

✓ Patient Added: John Doe (ID: 1) - ● LOW
✓ Patient Added: Sarah Smith (ID: 2) - ● CRITICAL
✓ Patient Added: Mike Johnson (ID: 3) - ● MEDIUM
✓ Patient Added: Emily Brown (ID: 4) - ● HIGH
✓ Patient Added: David Wilson (ID: 5) - ● CRITICAL

■ EMERGENCY QUEUE (Sorted by Priority):
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL | Time: 09:05
ID: 5 | Name: David Wilson | Severity: CRITICAL | Time: 09:20
ID: 4 | Name: Emily Brown | Severity: HIGH | Time: 09:15
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Time: 09:10
ID: 1 | Name: John Doe | Severity: LOW | Time: 09:00
-----

● Checking Next Patient:

■ Next Patient: Sarah Smith (Severity: CRITICAL)

■ TREATMENT PROCESS:

■ Now Treating: Sarah Smith (Severity: CRITICAL)
■ Now Treating: David Wilson (Severity: CRITICAL)
■ Now Treating: Emily Brown (Severity: HIGH)

■ EMERGENCY QUEUE (Sorted by Priority):
-----
ID: 3 | Name: Mike Johnson | Severity: MEDIUM | Time: 09:10
ID: 1 | Name: John Doe | Severity: LOW | Time: 09:00
-----

✓ TREATED PATIENTS:
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
-----

Treating remaining patients:

■ Now Treating: Mike Johnson (Severity: MEDIUM)
■ Now Treating: John Doe (Severity: LOW)

✓ TREATED PATIENTS:
-----
ID: 2 | Name: Sarah Smith | Severity: CRITICAL
ID: 5 | Name: David Wilson | Severity: CRITICAL
ID: 4 | Name: Emily Brown | Severity: HIGH
ID: 3 | Name: Mike Johnson | Severity: MEDIUM
ID: 1 | Name: John Doe | Severity: LOW
-----

PS C:\Users\arell\Music\aiac> 
```

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```

2024 """
2025 FEATURE MAPPING TABLE:
2026 =====
2027 1. Complaint Handling → Queue
2028     Justification: Complaints must be addressed in FIFO order based on arrival time.
2029     Queue ensures fair service and maintains chronological order without bias.
2030
2031 2. Fault Priority Handling → Priority Queue
2032     Justification: Critical faults need expedited repair regardless of arrival time.
2033     Priority queue sorts by severity level to address urgent faults first.
2034
2035 3. Consumer Record Lookup → Hash Table
2036     Justification: Provides O(1) average lookup time using consumer ID as key.
2037     Enables instant retrieval of consumer details, billing info, and account status.
2038
2039 4. Power Grid Connectivity → Graph
2040     Justification: Substations are nodes and transmission lines are edges in the network.
2041     Graph structure enables pathfinding and connectivity analysis for grid management.
2042
2043 5. Meter Reading History Tracking → Linked List
2044     Justification: Maintains chronological order of meter readings with easy insertion.
2045     Linked list efficiently stores historical data without requiring contiguous memory.
2046 """
2047
2048 # IMPLEMENTATION: Complaint Handling System using Queue
2049
2050 class Complaint:
2051
2052     def __init__(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
2053         self.complaint_id = complaint_id
2054         self.consumer_id = consumer_id
2055         self.consumer_name = consumer_name
2056         self.issue = issue
2057         self.reported_time = reported_time
2058         self.status = "Pending"
2059
2060     def __repr__(self):
2061         return f"Complaint({self.complaint_id}, {self.consumer_name}, {self.issue})"
2062
2063
2064 class ComplaintHandlingQueue:
2065
2066     def __init__(self):
2067         self.queue = []
2068         self.resolved = []
2069
2070     def register_complaint(self, complaint_id, consumer_id, consumer_name, issue, reported_time):
2071         complaint = Complaint(complaint_id, consumer_id, consumer_name, issue, reported_time)
2072         self.queue.append(complaint)
2073         print(f"✅ Complaint registered: ID {complaint_id} - {consumer_name} ({issue})")
2074
2075     def resolve_complaint(self):
2076         if not self.is_empty():
2077             complaint = self.queue.pop(0)
2078             complaint.status = "Resolved"
2079             self.resolved.append(complaint)
2080             print(f"✔ Resolved: Complaint {complaint.complaint_id} - {complaint.consumer_name}")
2081             return complaint
2082         else:
2083             print("⚠ No complaints in queue.")
2084             return None

```

```

2084         return None
2085
2086     def peek_next_complaint(self):
2087         if not self.is_empty():
2088             complaint = self.queue[0]
2089             print(f"🔍 Next complaint: ID {complaint.complaint_id} - {complaint.consumer_name} ({complaint.issue})")
2090             return complaint
2091         else:
2092             print("⚠️ Queue is empty.")
2093             return None
2094
2095     def is_empty(self):
2096         return len(self.queue) == 0
2097
2098     def get_queue_length(self):
2099         return len(self.queue)
2100
2101     def display_pending_complaints(self):
2102         if self.is_empty():
2103             print("📭 No pending complaints.\n")
2104         else:
2105             print("\n⚡ PENDING COMPLAINTS QUEUE (FIFO):")
2106             print("-" * 100)
2107             for i, complaint in enumerate(self.queue, 1):
2108                 print(f"{i}. ID: {complaint.complaint_id:<8} | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30} | Time: {complaint.reported_time}")
2109             print("-" * 100 + "\n")
2110
2111     def display_resolved_complaints(self):
2112         if not self.resolved:
2113             print("📭 No resolved complaints yet.\n")
2114         else:
2115             print("\n✅ RESOLVED COMPLAINTS:")
2116             print("-" * 100)
2117             for i, complaint in enumerate(self.resolved, 1):
2118                 print(f"{i}. ID: {complaint.complaint_id:<8} | Consumer: {complaint.consumer_name:<20} | Issue: {complaint.issue:<30}")
2119             print("-" * 100 + "\n")
2120
2121

```

```

2122 # Example Usage
2123 if __name__ == "__main__":
2124     print("=" * 100)
2125     print("⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE")
2126     print("=" * 100 + "\n")
2127
2128     system = ComplaintHandlingQueue()
2129
2130     # Register complaints
2131     print("📝 Registering complaints:\n")
2132     system.register_complaint(1001, 5001, "Raj Kumar", "Power outage", "09:00 AM")
2133     system.register_complaint(1002, 5002, "Priya Singh", "High billing", "09:15 AM")
2134     system.register_complaint(1003, 5003, "Amit Patel", "Meter malfunction", "09:30 AM")
2135     system.register_complaint(1004, 5004, "Sneha Gupta", "Damaged pole", "09:45 AM")
2136     system.register_complaint(1005, 5005, "Vikram Shah", "Connection issue", "10:00 AM")
2137
2138     # Display pending complaints
2139     system.display_pending_complaints()
2140
2141     # Peek at next complaint
2142     print("🔍 Checking next complaint:\n")
2143     system.peak_next_complaint()
2144     print(f"📊 Total Pending Complaints: {system.get_queue_length()}\n")
2145
2146     # Resolve complaints
2147     print("🔄 RESOLVING COMPLAINTS:\n")
2148     system.resolve_complaint()
2149     system.resolve_complaint()
2150     system.resolve_complaint()
2151
2152     # Display updated queue
2153     system.display_pending_complaints()
2154
2155     # Display resolved complaints
2156     system.display_resolved_complaints()
2157
2158     # Resolve remaining complaints
2159     print("Resolving remaining complaints:\n")
2160     system.resolve_complaint()
2161     system.resolve_complaint()
2162
2163     system.display_pending_complaints()
2164     system.display_resolved_complaints()
2165
2166     print("✅ All complaints have been resolved!")

```

Output:

```
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_11.py"
```

```
=====
⚡ SMART ELECTRICITY DISTRIBUTION SYSTEM - COMPLAINT HANDLING QUEUE
=====
```

```
📁 Registering complaints:
```

- ✅ Complaint registered: ID 1001 - Raj Kumar (Power outage)
- ✅ Complaint registered: ID 1002 - Priya Singh (High billing)
- ✅ Complaint registered: ID 1003 - Amit Patel (Meter malfunction)
- ✅ Complaint registered: ID 1004 - Sneha Gupta (Damaged pole)
- ✅ Complaint registered: ID 1005 - Vikram Shah (Connection issue)

```
⚡ PENDING COMPLAINTS QUEUE (FIFO):
```

```
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage | Time: 09:00 AM
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing | Time: 09:15 AM
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction | Time: 09:30 AM
4. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole | Time: 09:45 AM
5. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue | Time: 10:00 AM
-----
```

```
🔍 Checking next complaint:
```

```
📄 Next complaint: ID 1001 - Raj Kumar (Power outage)
```

```
📊 Total Pending Complaints: 5
```

```
🔧 RESOLVING COMPLAINTS:
```

- ✅ Resolved: Complaint 1001 - Raj Kumar
- ✅ Resolved: Complaint 1002 - Priya Singh
- ✅ Resolved: Complaint 1003 - Amit Patel

```
⚡ PENDING COMPLAINTS QUEUE (FIFO):
```

```
-----
1. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole | Time: 09:45 AM
2. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue | Time: 10:00 AM
-----
```

```
✅ RESOLVED COMPLAINTS:
```

```
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction
-----
```

```
Resolving remaining complaints:
```

- ✅ Resolved: Complaint 1004 - Sneha Gupta
- ✅ Resolved: Complaint 1005 - Vikram Shah
- 🔕 No pending complaints.

```
✅ RESOLVED COMPLAINTS:
```

```
-----
1. ID: 1001 | Consumer: Raj Kumar | Issue: Power outage
2. ID: 1002 | Consumer: Priya Singh | Issue: High billing
3. ID: 1003 | Consumer: Amit Patel | Issue: Meter malfunction
4. ID: 1004 | Consumer: Sneha Gupta | Issue: Damaged pole
5. ID: 1005 | Consumer: Vikram Shah | Issue: Connection issue
-----
```

```
✅ All complaints have been resolved!
```

```
PS C:\Users\arell\Music\aiac> █
```